



第5章 语法制导翻译技术



wenshli@bupt.edu.cn

知识点：语法制导定义、翻译方案

S-属性定义、L-属性定义

S-属性定义的翻译

L-属性定义的翻译

教学目标与要求

- 理解综合属性和继承属性；
- 掌握语法制导定义/翻译方案的设计思路和方法；
- 能够根据翻译需求，基于上下文无关文法设计相应的语法制导定义或语法制导翻译方案。
- 根据语法制导定义/翻译方案翻译输入符号串，验证设计结果的准确性。
- 理解LR分析翻译程序的实现方法；
理解S属性定义和L属性定义的LR实现。
- 能够根据翻译目标，分析翻译需求，设计所需要的属性、语义规则、设计语法制导定义/语法制导翻译方案，能够通过实例验证所设计方案的合理性和有效性，利用所设计语法制导定义或翻译方案翻译输入符号串得到翻译结果。

本章内容

语法制导翻译概述

5.1 语法制导定义及翻译方案

5.2 S-属性定义的自底向上翻译

5.3 L-属性定义的自顶向下翻译 (*)

5.4 L-属性定义的自底向上翻译

5.5 通用的语法制导翻译方法 (*)

小结

语法制导翻译概述

- 语义分析涉及到语言的语义
- 形式语义学的研究开始于20世纪60年代初，可以分为四类：
 - **操作语义学**：通过语言的实现方式（即语言成分所对应的计算机的操作）定义语言成分的语义，侧重模拟数据加工过程中计算机系统的操作。
 - **指称语义学**：通过执行语言成分所得到的最终效果来定义该语言成分的语义，侧重描述数据加工的结果，而非加工过程的细节。
 - **代数语义学**：用代数公理刻画语言成分的语义，主要研究抽象数据类型的代数规范，指称语义学的一个分支。
 - **公理语义学**：采用公理化方法描述程序对数据的加工，用公理系统定义程序设计语言的语义，另外，还研究和寻求适用于描述程序语义、便于语义推导的逻辑语言。
- 语法制导翻译技术
 - 多数编译程序普遍采用的一种技术
 - 比较接近形式化

语法制导翻译的整体思路

- 首先，根据翻译目标来确定每个产生式的语义；
 - 其次，根据产生式的含义，分析每个符号的语义；
 - 再次，把这些语义以属性的形式附加到相应的文法符号上（即把语义和语言结构联系起来）；
 - 然后，根据产生式的语义给出符号属性的求值规则（即语义规则），从而形成语法制导定义。
-
- 翻译：
根据语法分析过程中所使用的产生式，执行与之相应的语义规则，完成符号属性值的计算，从而完成翻译。

语法制导翻译示例

$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit.val}$

例如：考虑算术表达式文法

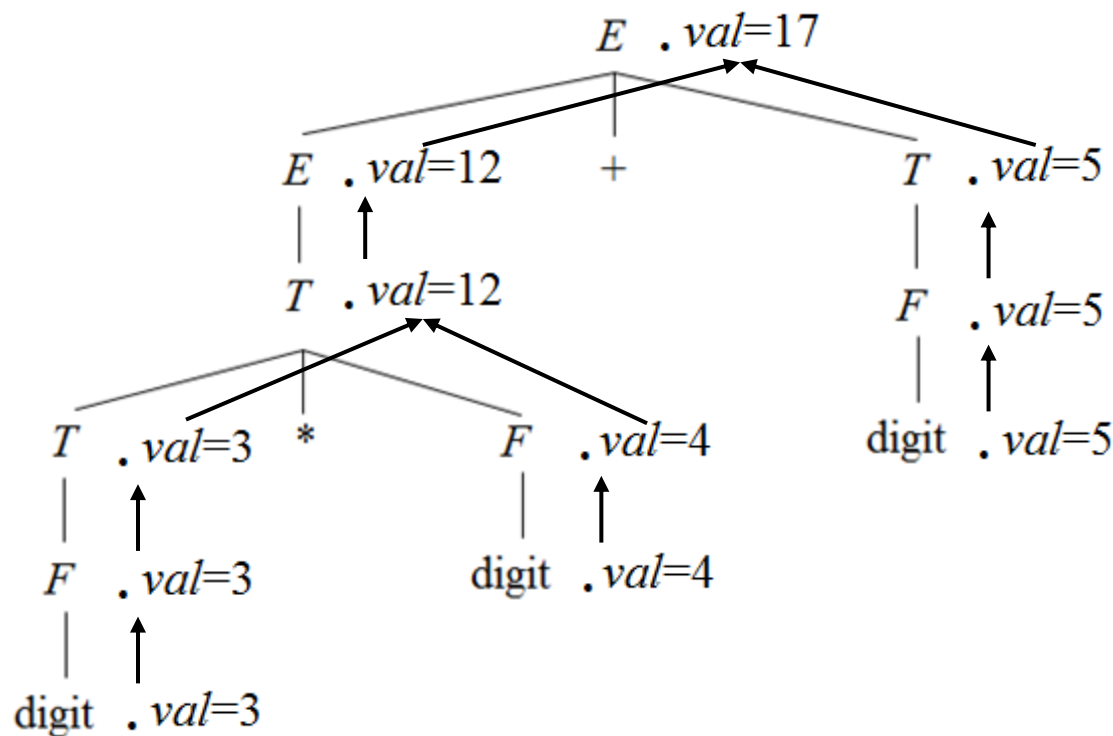
- 翻译目标：计算表达式的值
- 根据翻译目标确定每个产生式的语义；
 - $E \rightarrow E_1 + T$ ：表达式的值由两个子表达式的值相加得到
 - $F \rightarrow \text{digit}$ ：表达式的值即数字的值
- 根据产生式的语义，分析每个符号的语义；
 - E 、 T 、 F 、 digit 、 $+$ 、 $*$ 、 $($ 、 $)$
- 把这些语义以属性的形式附加到相应的文法符号上；
 - $E.val$ 、 $T.val$ 、 $F.val$ 、 digit.val
- 根据产生式的语义，给出符号属性的求值规则(即语义规则)，从而形成语法制导定义。
 - $E \rightarrow E_1 + T$ 对应的求值规则： $E.val = E_1.val + T.val$
 - 语法制导定义：产生式 语义规则

语法制导翻译示例

- 计算表达式 $3*4+5$ 的值
- 分析树:

$E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow T_1 * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow \text{digit}$

$E.\text{val} = E_1.\text{val} + T.\text{val}$
 $E.\text{val} = T.\text{val}$
 $T.\text{val} = T_1.\text{val} * F.\text{val}$
 $T.\text{val} = F.\text{val}$
 $F.\text{val} = E.\text{val}$
 $F.\text{val} = \text{digit}.\text{val}$



语法制导翻译示例

 $E \rightarrow E_1 + T$ $E.val = E_1.val + T.val$ $E \rightarrow T$ $E.val = T.val$ $T \rightarrow T_1 * F$ $T.val = T_1.val * F.val$ $T \rightarrow F$ $T.val = F.val$ $F \rightarrow (E)$ $F.val = E.val$ $F \rightarrow \text{digit}$ $F.val = \text{digit.val}$

例如：考虑算术表达式文法

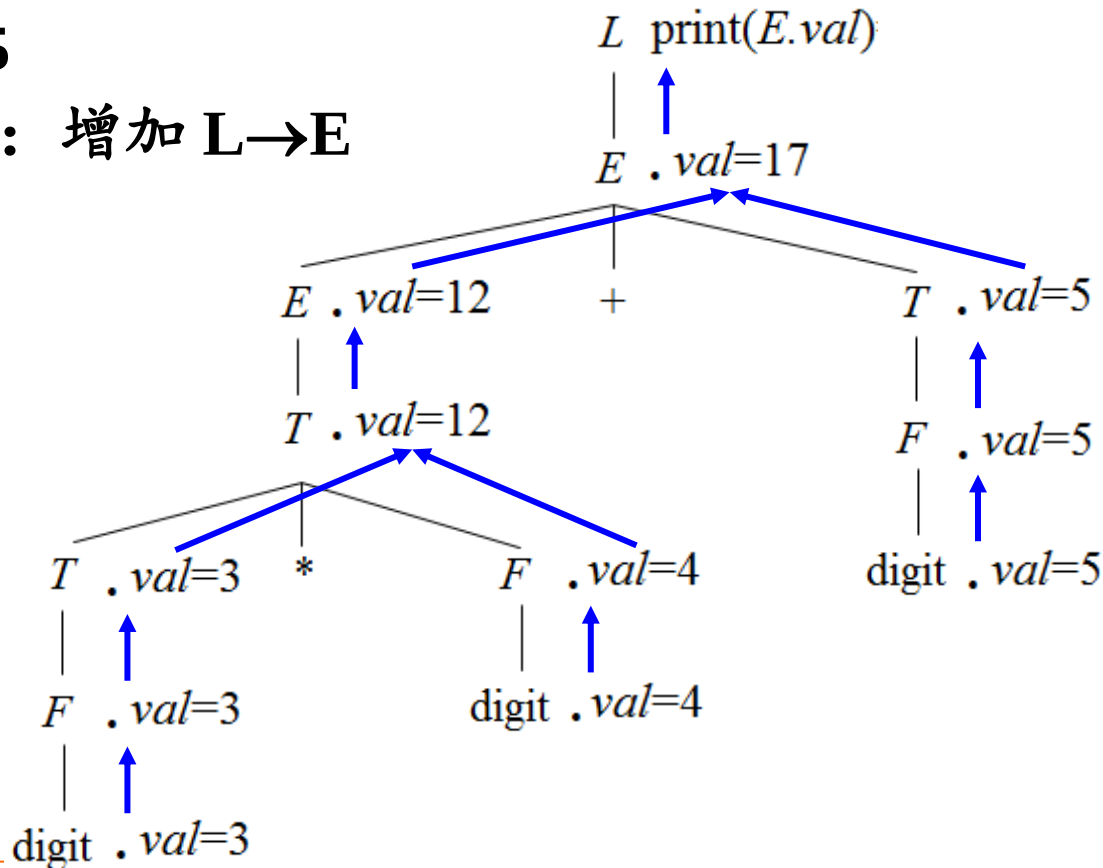
■ 翻译目标：计算并打印表达式的值

■ 根据语法分析过程中使用的产生式，执行相应的语义规则，完成相应的属性求值，从而完成翻译。

■ 例如：3*4+5

□ 拓广文法：增加 $L \rightarrow E$

□ 分析树：



翻译目标决定语义规则

- 翻译目标决定产生式的含义、决定文法符号应该具有的属性，也决定了产生式的语义规则。
- 例如：考虑算术表达式文法
 - 翻译目标：检查表达式的类型
 - $E \rightarrow E_1 + T$ 的语义：表达式的类型由两个子表达式的类型综合得到
 - 分析每个符号的语义，并以属性的形式记录：E.type、 $E_1.type$ 、T.type
 - 求值规则：
if ($E_1.type == integer$) && ($T.type == integer$)
 E.type=integer;
else ...

翻译结果依赖于语义规则

■ 翻译目标

□ 生成代码

- 可以为源程序产生中间代码
- 可以直接生成目标机指令

□ 对输入符号串进行解释执行

□ 向符号表中存放信息

□ 给出错误信息

■ 翻译的结果依赖于语义规则

□ 使用语义规则进行计算所得到的结果就是对输入符号串进行翻译的结果。

□ 如： $E \rightarrow E+T$ 的翻译结果可以是： 计算表达式的值、检查表达式的类型是否合法、 为表达式创建语法树、生成代码等等。

语法制导翻译的一般步骤

输入符号串

—————> 分析树

—————> 依赖图

—————> 语义规则的计算顺序

—————> 计算结果

语义规则的执行时机

- 可以用一个或多个子程序（称为**语义动作**）所要完成的功能描述产生式的语义。
- 在**语法分析过程中**使用某个产生式时，在**适当的时机**执行相应的语义动作，完成所需要的翻译。
- 把语义动作**插入到产生式中适当的位置**，从而形成**翻译方案**。
- 语法制导定义是对翻译的高层次的说明，它隐蔽了一些实现细节，无须指明翻译时语义规则的计算次序。
- 翻译方案指明了语义规则的计算次序，规定了语义动作的执行时机。

5.1 语法制导定义及翻译方案

5.1.1 语法制导定义

5.1.2 依赖图

5.1.3 计算次序

5.1.4 S属性定义和L属性定义

5.1.5 翻译方案

5.1.1 语法制导定义

- 对上下文无关文法的推广
- 每个文法符号都可以有一个**属性集**，其中可以包括两类属性：**综合属性**和**继承属性**。
 - **左部符号的综合属性**是从该产生式右部文法符号的属性值计算出来的；在分析树中，一个内部结点的**综合属性**是从其子结点的属性值计算出来的。
 - 出现在产生式**右部的某文法符号的继承属性**是从其所在产生式的左部非终结符号和/或右部文法符号的属性值计算出来的；在分析树中，一个结点的**继承属性**是从其兄弟结点和/或父结点的属性值计算出来的。
 - 分析树中某个结点的**属性值**是由与在这个结点上所用产生式相应的**语义规则**决定的。
- 和产生式相联系的**语义规则**建立了属性之间的关系，这些关系可用**有向图**（即：**依赖图**）来表示。

语法制导定义

- 在一个语法制导定义中，对应于每一个文法产生式 $A \rightarrow \alpha$ ，都有与之相联系的一组语义规则，其形式为：
$$b = f(c_1, c_2, \dots, c_k)$$

这里， f 是一个函数，而且

- (1) 如果 b 是 A 的一个综合属性，则 c_1 、 c_2 、...、 c_k 是产生式右部文法符号的属性或者 A 的继承属性；
 - (2) 如果 b 是产生式右部某个文法符号的一个继承属性，则 c_1 、 c_2 、...、 c_k 是 A 或产生式右部任何文法符号的属性。
- 属性 b 依赖于属性 c_1 、 c_2 、...、 c_k 。
 - 语义规则函数都不具有副作用的语法制导定义称为属性文法

语义规则

■ 一般情况：

- 语义规则函数可写成表达式的形式。
- 比如： $E.val = E_1.val + T.val$

■ 某些情况下：

- 一个语义规则的唯一目的就是产生某个副作用（完成一个特定的功能），如打印一个值、向符号表中插入一条记录等；
- 这样的语义规则通常写成过程调用或程序段。
- 比如： `print(E.val)`
- 看成是相应产生式左部非终结符号的虚拟综合属性。
- 虚属性和符号 ‘=’ 都没有表示出来。

简单算术表达式求值的语法制导定义

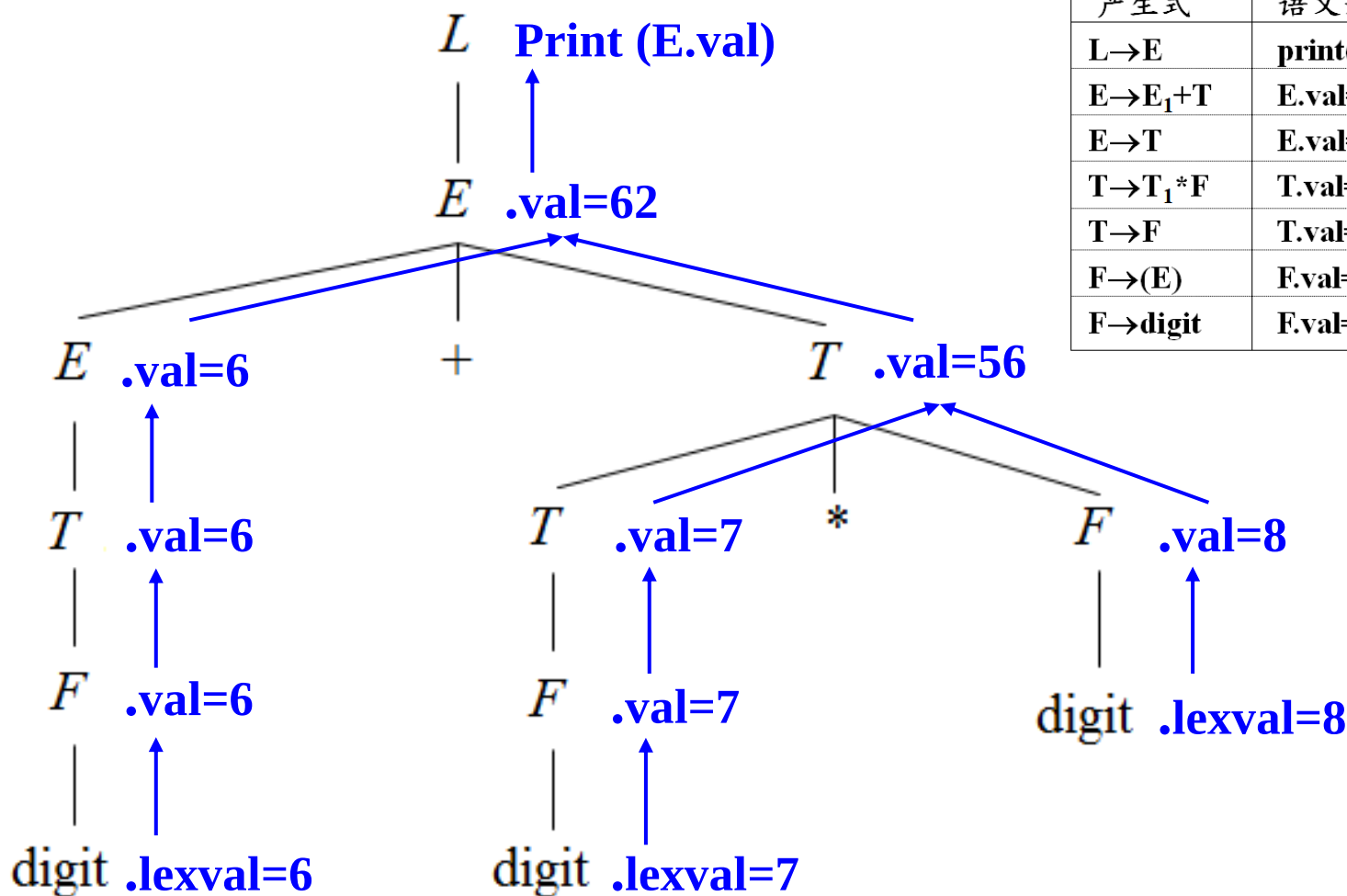
产生式	语义规则
$L \rightarrow E$	print(E.val)
$E \rightarrow E_1 + T$	E.val = E₁.val + T.val
$E \rightarrow T$	E.val = T.val
$T \rightarrow T_1 * F$	T.val = T₁.val * F.val
$T \rightarrow F$	T.val = F.val
$F \rightarrow (E)$	F.val = E.val
$F \rightarrow \text{digit}$	F.val = digit.lexval

- 每一个非终结符号E、T和F都有一个**综合属性** val
- 表示相应非终结符号所代表的子表达式的值
- $L \rightarrow E$ 的语义规则是一个过程，左部符号**L**的**虚拟综合属性**，打印出由E产生的算术表达式的值。

综合属性

- 分析树中，如果一个结点的某一属性由其子结点的属性确定，则这种属性为该结点的**综合属性**。
- 如果一个语法制导定义仅仅使用综合属性，则称这种语法制导定义为**S-属性定义**。
- 对于S-属性定义，通常采用**自底向上**的方法对其分析树加注释，即从树叶到树根，按照语义规则计算每个结点的属性值。
- 简单计算器的语法制导定义是S-属性定义

6+7*8的分析树加注释的过程



产生式	语义规则
$L \rightarrow E$	print (E.val)
$E \rightarrow E_1 + T$	E.val =E ₁ .val+T.val
$E \rightarrow T$	E.val =T.val
$T \rightarrow T_1 * F$	T.val =T ₁ .val*F.val
$T \rightarrow F$	T.val =F.val
$F \rightarrow (E)$	F.val =E.val
$F \rightarrow \text{digit}$	F.val =digit.lexval

■ 属性值的计算可以在语法分析过程中进行。

继承属性

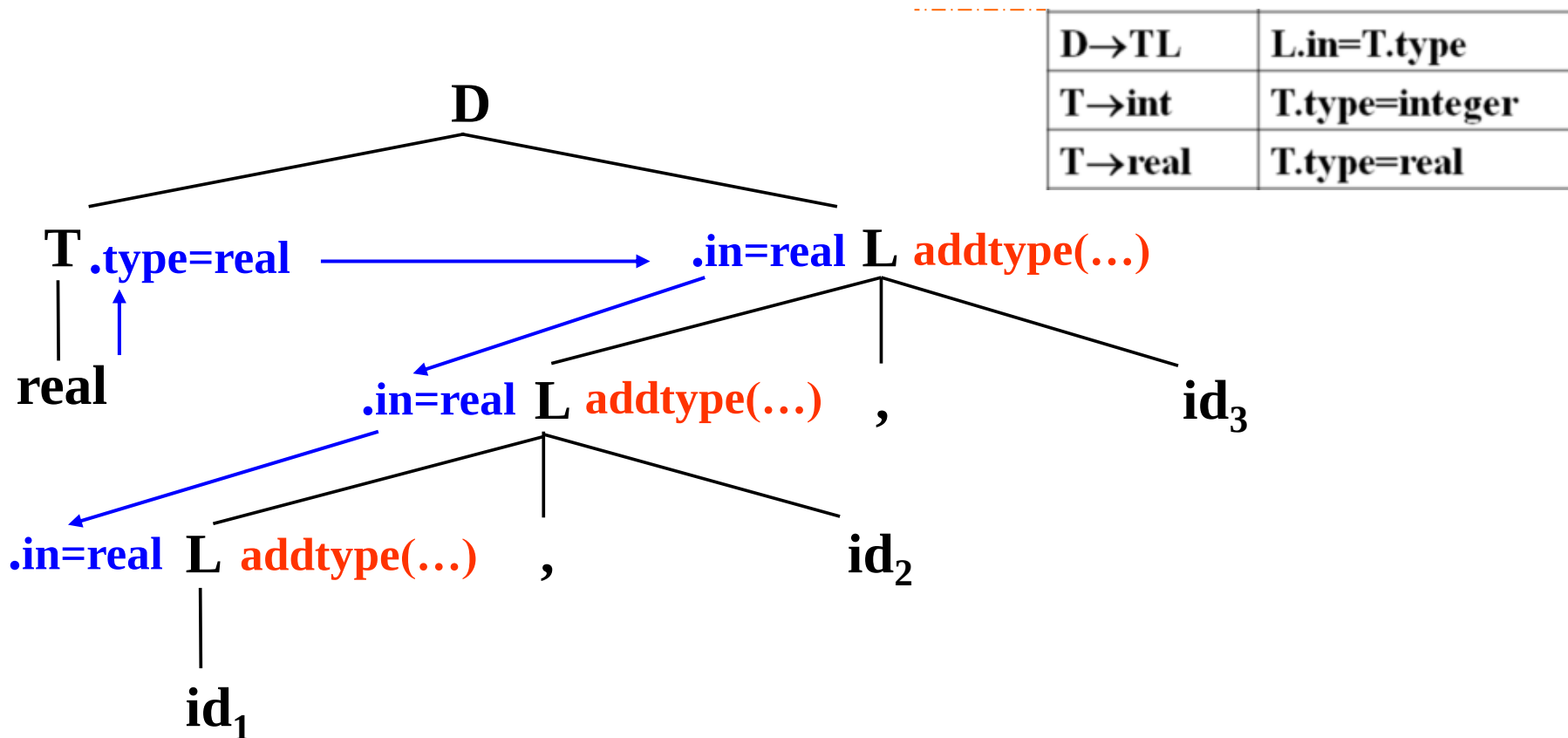
- 分析树中，一个结点的继承属性值由该结点的父结点和/或它的兄弟结点的属性值决定。
- 可用继承属性表示程序设计语言结构中上下文之间的依赖关系
 - 可以跟踪一个标识符的类型
 - 可以跟踪一个标识符，了解它是出现在赋值号的右边还是左边，以确定是需要该标识符的值还是地址。

用继承属性L.in传递类型信息的语法制导定义

产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in;$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

- D产生的声明语句包含了类型关键字int或real，后跟一个标识符表。
- T有综合属性type，其值由声明中的关键字确定。
- L的继承属性L.in，
 - 产生式 $D \rightarrow TL$ L.in 表示从其兄弟结点T继承下来的类型信息。
 - 产生式 $L \rightarrow L_1, id$ $L_1.in$ 表示从其父结点L继承下来的类型信息

语句 $\text{real id}_1, \text{id}_2, \text{id}_3$ 的注释分析树



$L \rightarrow L_1, \text{id}$	$L_1.in = L.in;$ $\text{addtype}(\text{id.entry}, L.in)$
$L \rightarrow \text{id}$	$\text{addtype}(\text{id.entry}, L.in)$

把类型信息在分析树中向下传递

把类型信息填入符号表中标识符记录中

5.1.2 依赖图

- 分析树中，结点的继承属性和综合属性之间的相互依赖关系可以由依赖图表示。
- 为每个包含过程调用的语义规则引入一个**虚拟综合属性** b
 - 把语义规则统一为 $b=f(c_1, c_2, \dots, c_k)$ 的形式
- 依赖图中：
 - 为每个属性设置一个结点
 - 如果属性 b 依赖于 c ，那么从属性 c 的结点有一条有向边连到属性 b 的结点。

算法5.1 构造依赖图

产生式	语义规则
$A \rightarrow XY$	$A.a = f(X.x, Y.y)$ $X.i = g(A.a, Y.y)$

输入：一棵分析树，语法制导定义

输出：一张依赖图

方法：

for (分析树中每一个结点 n)

for (结点 n 处的文法符号的
每一个属性 a)

为 a 在依赖图中建立一个结点;

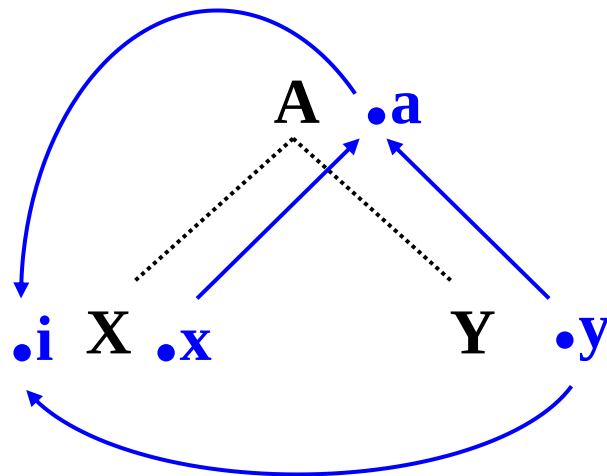
for (分析树中每一个结点 n)

for (结点 n 处所用产生式对应的每一个语义规则

$b = f(c_1, c_2, \dots, c_k)$)

for ($i=1; i \leq k; i++$)

从 c_i 结点到 b 结点构造一条有向边;

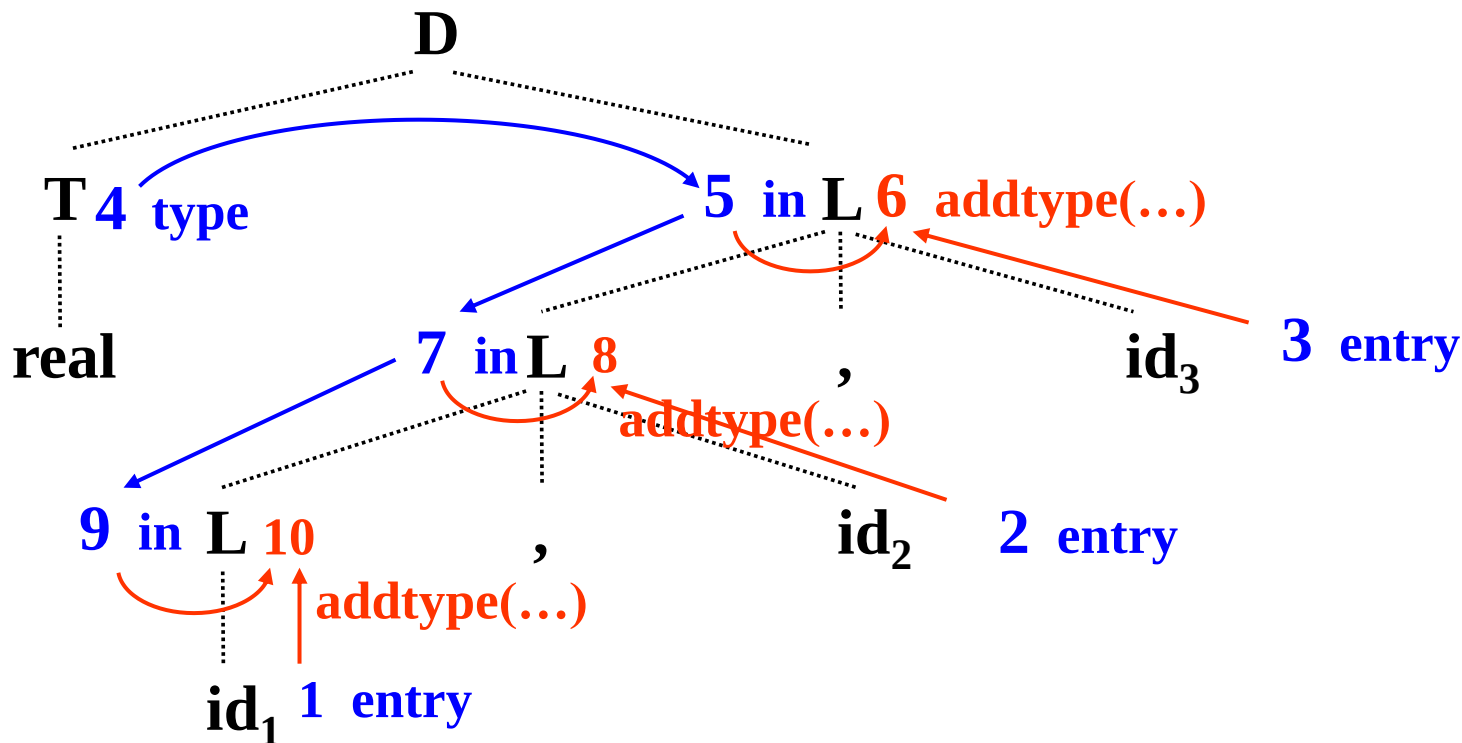


依赖图构造举例

real p, q, r

real $\text{id}_1, \text{id}_2, \text{id}_3$

产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in;$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$



5.1.3 计算次序

■ 有向非循环图的拓扑排序

- 图中结点的一种排序 $m_1, m_2, \dots, m_k$
- 有向边只能从这个序列中前边的结点指向后面的结点
- 如果 $m_i \rightarrow m_j$ 是从 m_i 指向 m_j 的一条边，那么在序列中 m_i 必须出现在 m_j 之前。

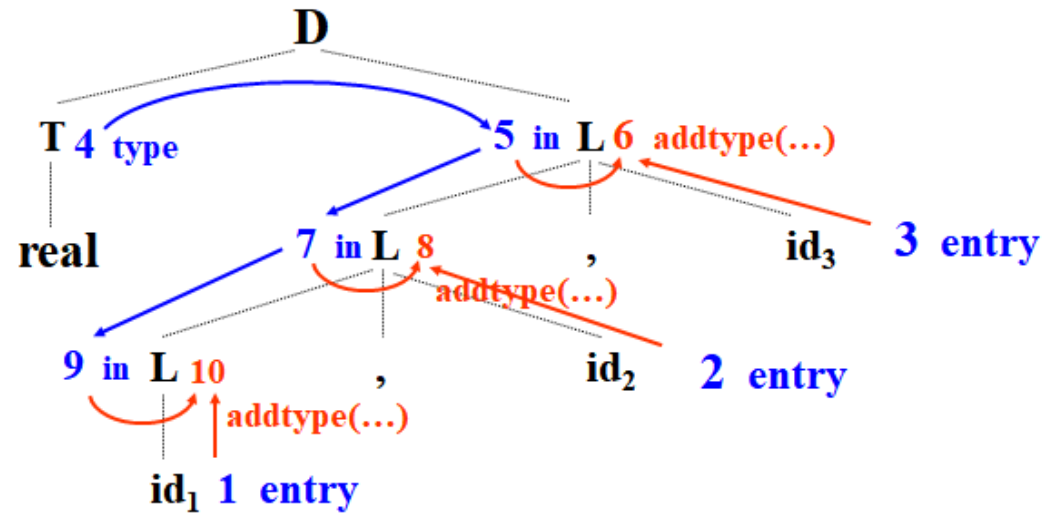
■ 依赖图的任何拓扑排序

- 给出了分析树中结点的语义规则计算的有效顺序
- 在拓扑排序中，一个结点上语义规则 $b=f(c_1, c_2, \dots, c_k)$ 中的属性 $c_1, c_2, \dots, c_k$ 在计算 b 时都是可用的。

■ 拓扑排序：

- 1、2、3、4、5、6、7、8、9、10
- 4、5、3、6、7、2、8、9、1、10

计算顺序



```
type=real;  
in5=type;  
addtype(id3.entry, in5);  
in7=in5;  
addtype(id2.entry, in7);  
in9=in7;  
addtype(id1.entry, in9);
```

```
type=real;  
in5=type;  
in7=in5;  
in9=in7;  
addtype(id1.entry, in9);  
addtype(id2.entry, in7);  
addtype(id3.entry, in5);
```

语法制导翻译过程

- 最基本的文法用于建立输入符号串的分析树；
- 为分析树构造依赖图；
- 对依赖图进行拓扑排序；
- 从这个序列得到语义规则的计算顺序；
- 照此计算顺序进行求值，得到对输入符号串的翻译。

输入符号串



分析树



依赖图



语义规则的计算顺序



计算结果

示例：翻译 $3*4+5$

$L \rightarrow E$	$\text{Print}(E.val)$
$E \rightarrow E_1 + T$	$E.val = E_1.val + T.val$
$E \rightarrow T$	$E.val = T.val$
$T \rightarrow T_1 * F$	$T.val = T_1.val * F.val$
$T \rightarrow F$	$T.val = F.val$
$F \rightarrow (E)$	$F.val = E.val$
$F \rightarrow \text{digit}$	$F.val = \text{digit}.val$

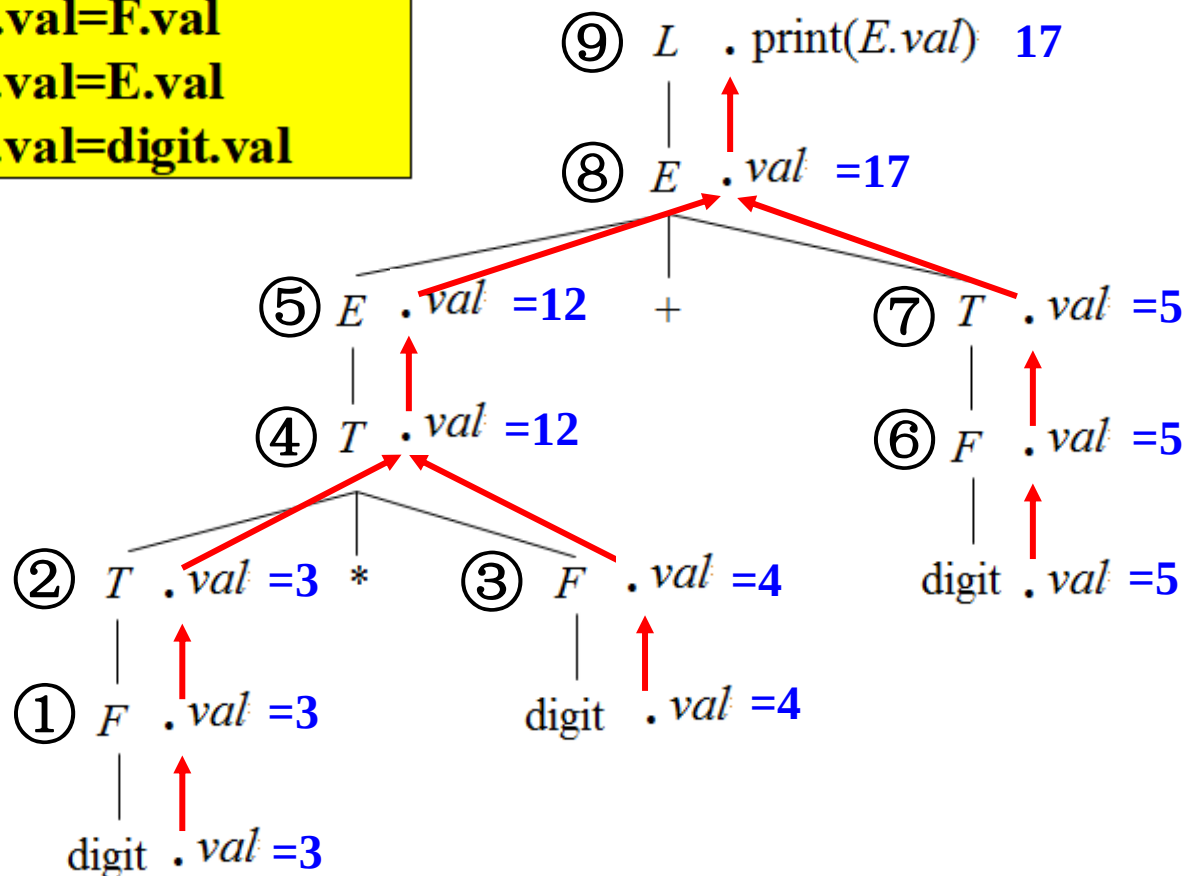
输入符号串

分析树

依赖图

语义规则的计算顺序

计算结果



5.1.4 S属性定义和L属性定义

- S属性定义：仅涉及综合属性的语法制导定义
- L属性定义：一个语法制导定义是L属性定义，如果与每个产生式 $A \rightarrow X_1 X_2 \dots X_n$ 相应的每条语义规则计算的属性都是：
 - A的综合属性，或是
 - X_j ($1 \leq j \leq n$) 的继承属性，而该继承属性仅依赖于：
 - A的继承属性；
 - 产生式中 X_j 左边的符号 X_1 、 X_2 、...、 X_{j-1} 的属性。
- 每一个S属性定义都是L属性定义

语法制导定义示例

L属性定义 →

产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in;$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

非L属性定义 →

产生式	语义规则
$A \rightarrow LM$	$L.i = l(A.i)$ $M.i = m(L.s)$ $A.s = f(M.s)$
$A \rightarrow QR$	$R.i = r(A.i)$ $Q.i = q(R.s)$ $A.s = f(Q.s)$

属性计算顺序—深度优先遍历分析树

```
void deepfirst (n: node)
{
    for (n的每一个子结点m, 从左到右) {
        计算m的继承属性;
        deepfirst(m);
    };
    计算n的综合属性;
}.
```

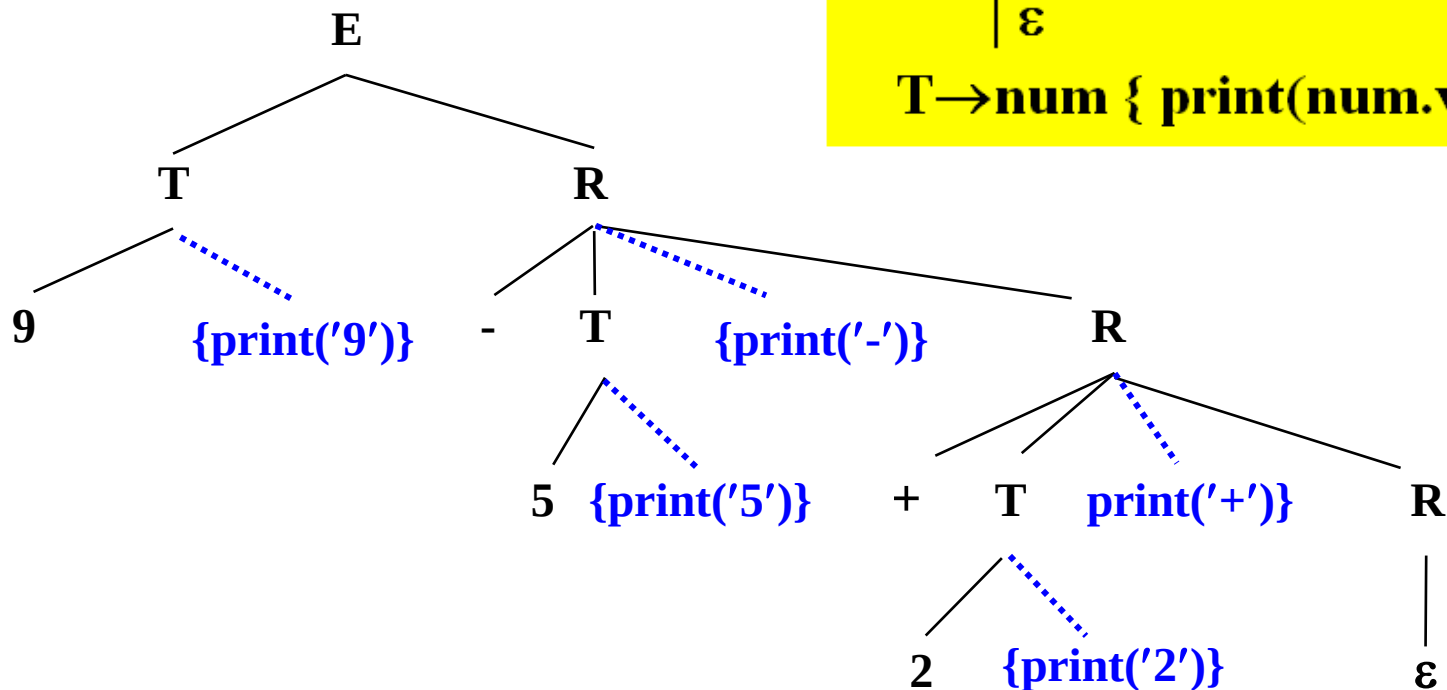
- 以分析树的根结点作为实参
- L属性定义的属性都可以用深度优先的顺序计算。
 - 进入结点前，计算它的继承属性
 - 从结点返回时，计算它的综合属性

5.1.5 翻译方案

- 上下文无关文法的一种便于翻译的书写形式
- 属性与文法符号相对应
- 语义动作括在花括号中，并插入到产生式右部某个合适的位置上
- 给出了使用语义规则进行属性计算的顺序
- 分析过程中翻译的注释

翻译方案示例

9-5+2的分析树:



翻译方案:

$E \rightarrow TR$

$R \rightarrow +T \{ \text{print}(' + ') \} R_1$

$\quad \mid -T \{ \text{print}(' - ') \} R_1$

$\quad \mid \varepsilon$

$T \rightarrow \text{num} \{ \text{print}(\text{num.val}) \}$

语义动作作为相应产生式左部符号对应结点的子结点
深度优先遍历树中结点, 执行其中的动作, 打印出95-2+

翻译方案的设计

■ 对于S属性定义：

- 为每一个语义规则建立一个包含赋值的动作
- 把这个动作放在相应的产生式右边末尾

例：产生式 语义规则

$T \rightarrow T_1 * F$ $T.val = T_1.val * F.val$

如下安排产生式和语义动作：

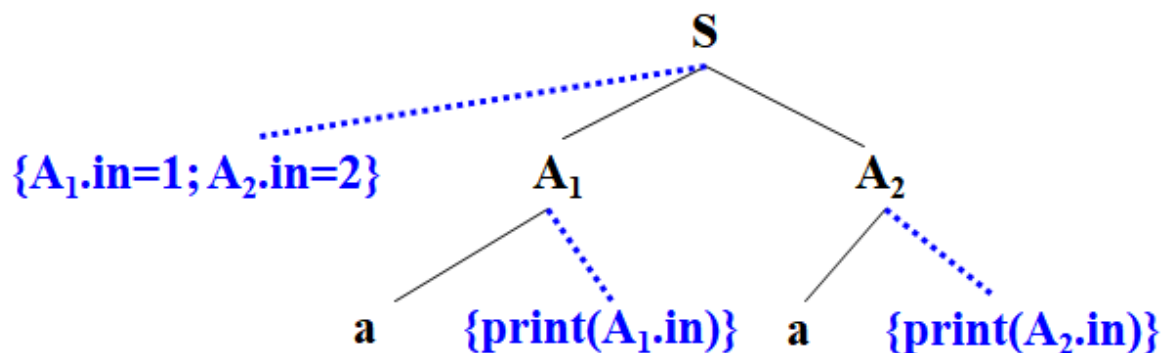
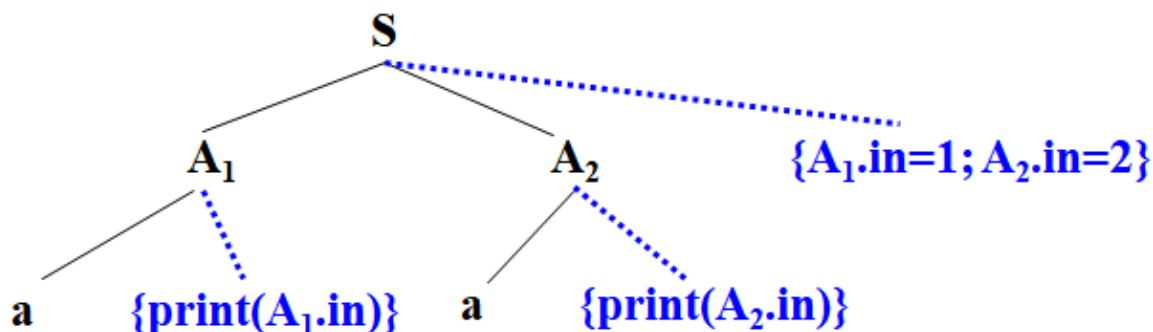
$T \rightarrow T_1 * F \{ T.val = T_1.val * F.val \}$

为L属性定义设计翻译方案的原则

- 一个动作不能引用这个动作右边的文法符号的综合属性
- 左部符号的**综合属性**只有在它所引用的所有属性都计算出来之后才能计算
 - 这种属性的计算动作放在产生式右端末尾
- 右部符号的**继承属性**必须在这个符号以前的动作中计算出来
 - 计算该继承属性的动作必须出现在相应文法符号之前

示例

考虑如下翻译方案：

$$S \rightarrow A_1 A_2 \{ A_1.in=1; A_2.in=2 \}$$
$$A \rightarrow a \{ \text{print}(A.in) \}$$


L属性定义翻译方案设计举例

■ 语法制导定义

产生式	语义规则
$D \rightarrow TL$	$L.in = T.type$
$T \rightarrow int$	$T.type = integer$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.in = L.in;$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

■ 翻译方案

$D \rightarrow T \{ L.in = T.type \} L$

$T \rightarrow int \{ T.type = integer \}$

$T \rightarrow real \{ T.type = real \}$

$L \rightarrow \{ L_1.in = L.in \} L_1, id \{ addtype(id.entry, L.in) \}$

$L \rightarrow id \{ addtype(id.entry, L.in) \}$

例：为如下L属性定义设计翻译方案

产生式	语义规则
$S \rightarrow B$	$B.ps = 10; \quad S.ht = B.ht$
$B \rightarrow B_1 B_2$	$B_1.ps = B.ps$ $B_2.ps = B.ps$ $B.ht = \max(B_1.ht, B_2.ht)$
$B \rightarrow B_1 \text{sub} B_2$	$B_1.ps = B.ps$ $B_2.ps = \text{shrink}(B.ps)$ $B.ht = \text{disp}(B_1.ht, B_2.ht)$
$B \rightarrow \text{text}$	$B.ht = \text{text.h} \times B.ps$

■ 唯一的继承属性B.ps

- 常数
- 依赖于左部B的B.ps

■ 翻译方案如下：

$S \rightarrow \{B.ps = 10\} B \{S.ht = B.ht\}$

$B \rightarrow \{B_1.ps = B.ps\} B_1 \{B_2.ps = B.ps\} B_2 \{B.ht = \max(B_1.ht, B_2.ht)\}$

$B \rightarrow \{B_1.ps = B.ps\} B_1 \text{sub} \{B_2.ps = \text{shrink}(B.ps)\} B_2$
 $\{B.ht = \text{disp}(B_1.ht, B_2.ht)\}$

$B \rightarrow \text{text} \{B.ht = \text{text.h} \times B.ps\}$

作业：

5.16

5.17



5.2 S-属性定义的自底向上翻译

■ S属性定义：

- 只用综合属性的语法制导定义

5.2.1 为表达式构造语法树的语法制导定义

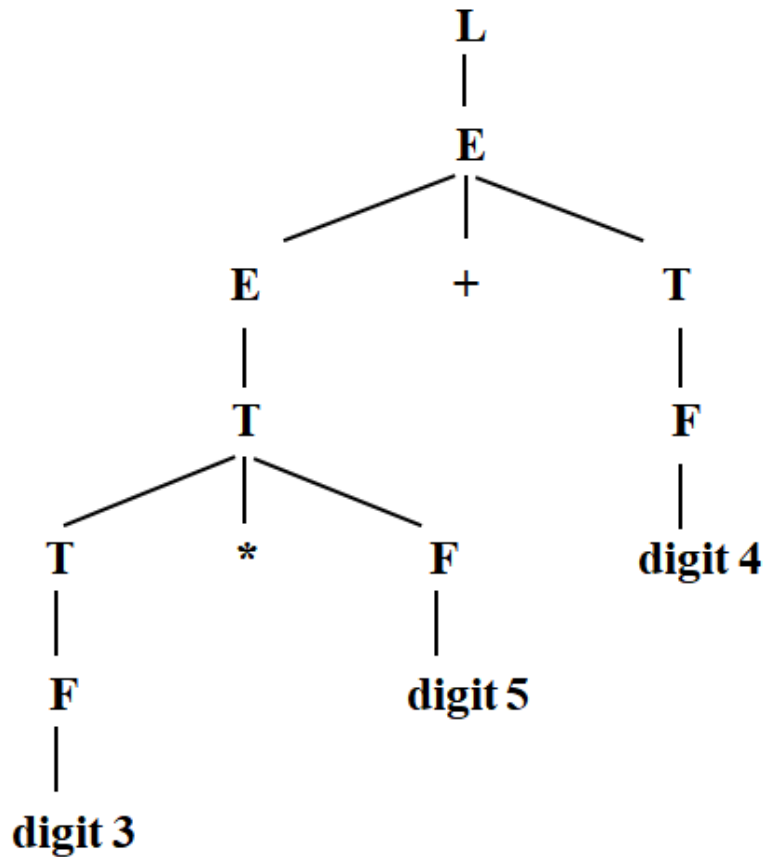
5.2.2 S属性定义的自底向上实现

5.2.1 为表达式构造语法树的语法制导定义

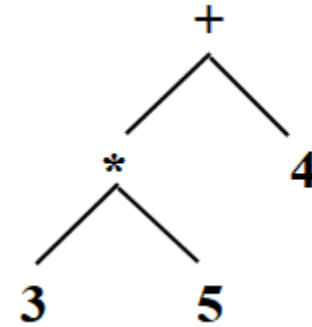
- 抽象语法：把语法规则中对语义无关紧要的具体规定去掉，剩下的本质性的东西称为抽象语法。
- 如：
 - 赋值语句： $x=y$ 、 $x:=y$ 、或 $y\rightarrow x$
 - 抽象形式：`assignment(variable, expression)`
- 语法树：
 - 分析树的抽象（或压缩）形式。
 - 也称为语法结构树或结构树。
 - 内部结点表示运算符号，其子结点表示它的运算分量。

语法树示例

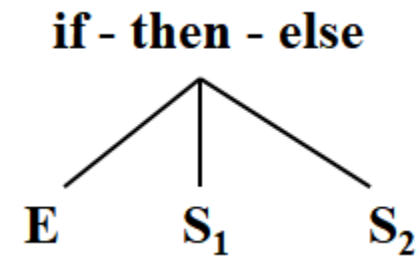
■ 表达式 $3*5+4$ 的分析树



表达式 $3*5+4$ 的语法树



■ $S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$ 的语法树



构造表达式的语法树

■ 表达式的语法树的形式

- 每一个运算符或运算分量都对应树中的一个结点
- 运算符结点的子结点分别是与该运算符的各个运算分量相应的子树的根。
- 每一个结点可包含若干个域：
 - 标识域、指针域、属性值域等

■ 在运算符结点中

- 一个域标识运算符
- 其它各域包含指向与各运算分量相应的结点的指针
- 称运算符为该结点的标号

构造函数

■ makenode (op, left, right)

- 建立一个运算符结点，标号是 op;
- 域left和right是指向其左右运算分量结点的指针。

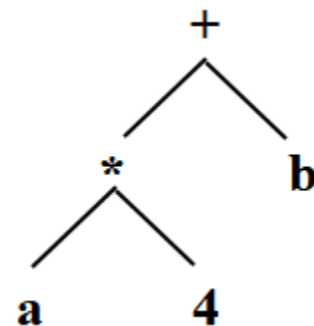
■ makeleaf (id, entry)

- 建立一个标识符结点，标号是 id;
- 域entry是指向该标识符在符号表中的相应条目的指针。

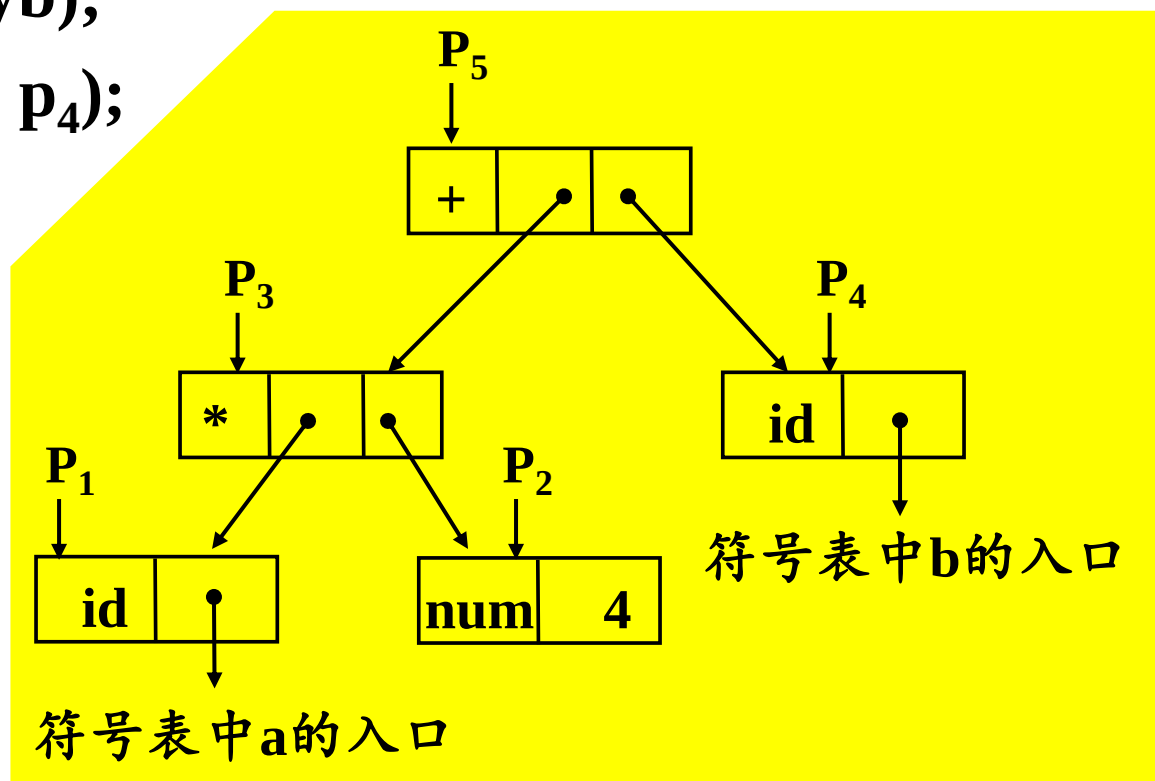
■ makeleaf (num, val)

- 建立一个数结点，标号为 num;
- 域val用于保存该数的值。

建立表达式 $a*4+b$ 的语法树



$p_1 = \text{makeleaf}(\text{id}, \text{entrya});$
 $p_2 = \text{makeleaf}(\text{num}, 4);$
 $p_3 = \text{makenode}('*', p_1, p_2);$
 $p_4 = \text{makeleaf}(\text{id}, \text{entryb});$
 $p_5 = \text{makenode}('+', p_3, p_4);$



构造表达式语法树的语法制导定义

- 翻译目标：为表达式创建语法树
- 产生式语义：创建与产生式左部符号代表的子表达式对应的子树，即创建子树的根结点。
- 文法符号的属性：记录所建结点， $E.nptr$ 、 $T.nptr$ 、 $F.nptr$ 指向相应子树根结点的指针
- 产生式的语义动作举例：

$E \rightarrow E_1 + T$ $E.nptr = \text{makenode}('+', E_1.nptr, T.nptr)$

$T \rightarrow F$ $T.nptr = F.nptr$

$F \rightarrow \text{id}$ $F.nptr = \text{makeleaf}(\text{id}, \text{id.entry})$

$F \rightarrow \text{num}$ $F.nptr = \text{makeleaf}(\text{num}, \text{num.val})$

构造表达式语法树的语法制导定义

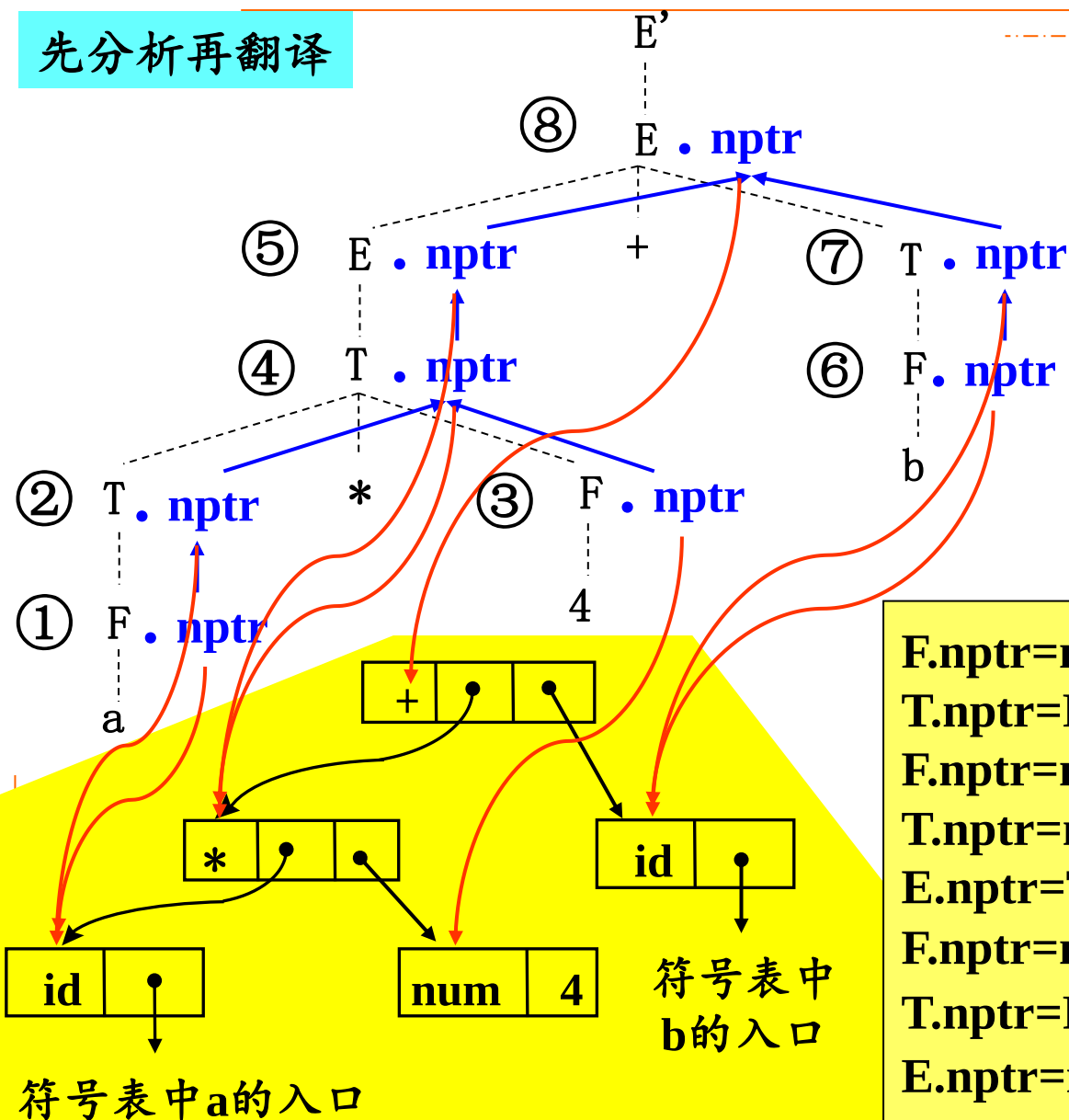
产生式	语义规则
$E \rightarrow E_1 + T$	$E.nptr = \text{makenode}('+', E_1.nptr, T.nptr)$
$E \rightarrow T$	$E.nptr = T.nptr$
$T \rightarrow T_1 * F$	$T.nptr = \text{makenode}('*', T_1.nptr, F.nptr)$
$T \rightarrow F$	$T.nptr = F.nptr$
$F \rightarrow (E)$	$F.nptr = E.nptr$
$F \rightarrow \text{id}$	$F.nptr = \text{makeleaf}(\text{id}, \text{id.entry})$
$F \rightarrow \text{num}$	$F.nptr = \text{makeleaf}(\text{num}, \text{num.val})$

- 为了记录在构造过程中建立的子树，为每个非终结符号引入一个综合属性 $nptr$ 。
- $nptr$ 是一个指针，指向语法树中相应非终结符号产生的表达式子树的根结点。

表达式a*4+b的语法树的构造

先分析再翻译

$E' \rightarrow E$
 $E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow T_1 * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$
 $F \rightarrow num$

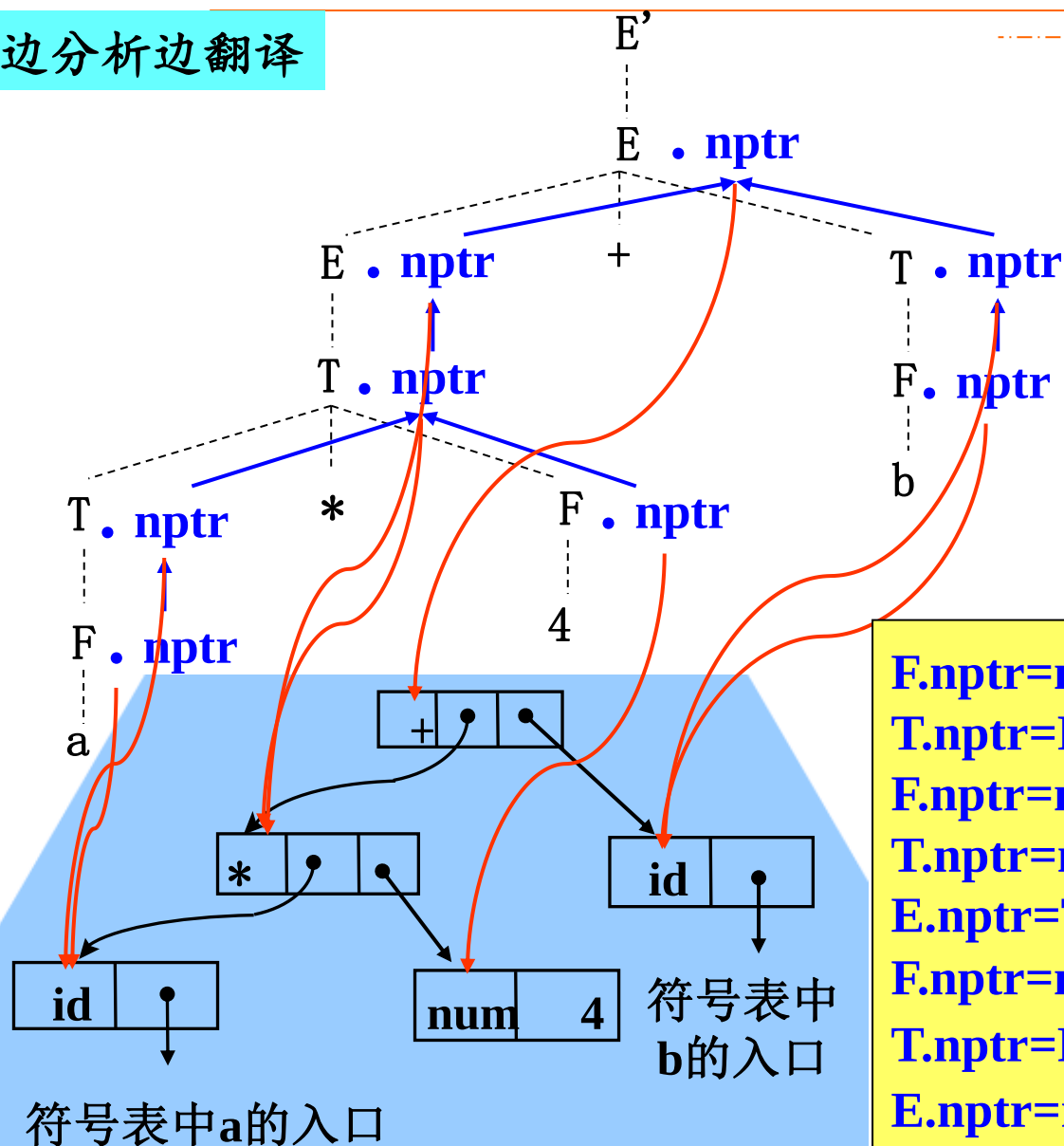


$F.nptr = makeleaf(id, entrya)$
 $T.nptr = F.nptr$
 $F.nptr = makeleaf(num, 4)$
 $T.nptr = makenode('*', T_1.nptr, F.nptr)$
 $E.nptr = T.nptr$
 $F.nptr = makeleaf(id, entryb)$
 $T.nptr = F.nptr$
 $E.nptr = makenode('+', E_1.nptr, T.nptr)$

表达式a*4+b的语法树的构造

边分析边翻译

$E' \rightarrow E$
 $E \rightarrow E_1 + T$
 $E \rightarrow T$
 $T \rightarrow T_1 * F$
 $T \rightarrow F$
 $F \rightarrow (E)$
 $F \rightarrow id$
 $F \rightarrow num$



$F.nptr = makeleaf(id, entrya)$
 $T.nptr = F.nptr$
 $F.nptr = makeleaf(num, 4)$
 $T.nptr = makenode('*', T_1.nptr, F.nptr)$
 $E.nptr = T.nptr$
 $F.nptr = makeleaf(id, entryb)$
 $T.nptr = F.nptr$
 $E.nptr = makenode('+', E_1.nptr, T.nptr)$

表达式的有向非循环图(dag)

■ dag与语法树相同的地方：

- 表达式的每一个子表达式都有一个结点
- 一个内部结点表示一个运算符，且它的子结点表示它的运算分量。

■ dag与语法树不同的地方：

- dag中，对应一个公共子表达式的结点具有多个父结点
- 语法树中，公共子表达式被表示为重复的子树

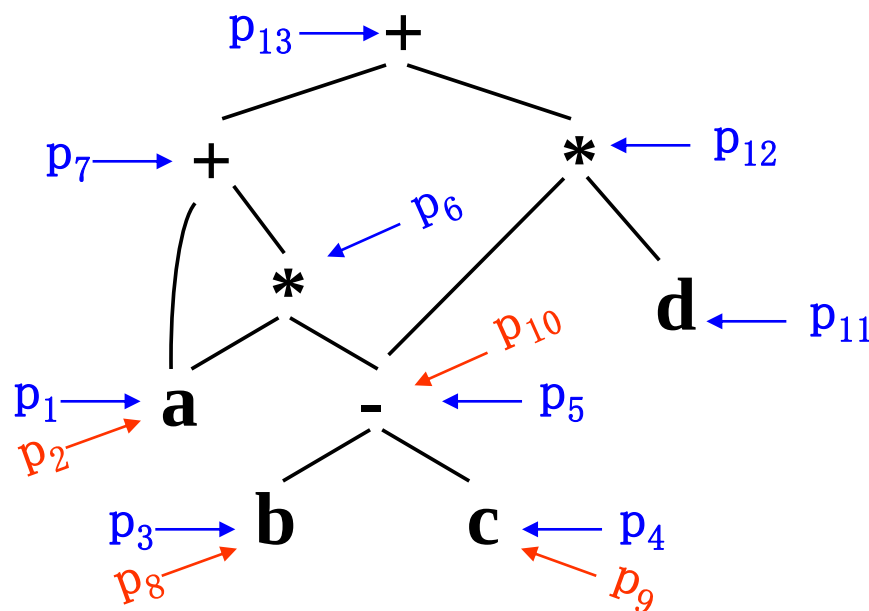
■ 为表达式创建dag的函数makenode和makeleaf

- 建立新结点之前先检查是否已经存在一个相同的结点
- 若已存在，返回一个指向先前已构造好的结点的指针；
- 否则，创建一个新结点，返回指向新结点的指针。

为表达式 $a+a*(b-c)+(b-c)*d$ 构造dag

函数调用

- $p_1 = \text{makeleaf}(\text{id}, a);$
- $p_2 = \text{makeleaf}(\text{id}, a);$
- $p_3 = \text{makeleaf}(\text{id}, b);$
- $p_4 = \text{makeleaf}(\text{id}, c);$
- $p_5 = \text{makenode}('-', p_3, p_4);$
- $p_6 = \text{makenode}('*', p_2, p_5);$
- $p_7 = \text{makenode}('+', p_1, p_6);$
- $p_8 = \text{makeleaf}(\text{id}, b);$
- $p_9 = \text{makeleaf}(\text{id}, c);$
- $p_{10} = \text{makenode}('-', p_8, p_9);$
- $p_{11} = \text{makeleaf}(\text{id}, d);$
- $p_{12} = \text{makenode}('*', p_{10}, p_{11});$
- $p_{13} = \text{makenode}('+', p_7, p_{12});$



5.2.2 S-属性定义的自底向上实现

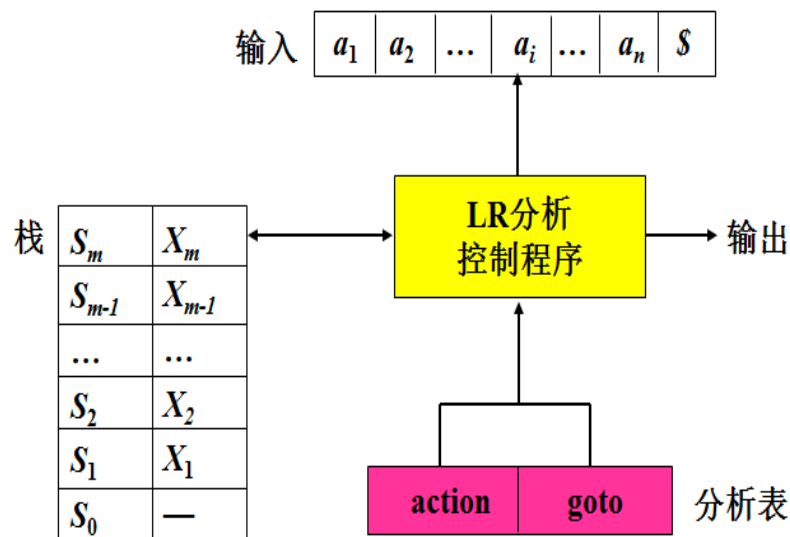
■ LR分析程序的模型

■ 已知

- LR分析方法中，分析程序使用**栈**来存放已经分析过的子树的信息。
- 分析树中某结点的**综合属性**由其子结点的属性值计算得到。
- LR分析程序在分析输入符号串的**同时**可以计算综合属性

■ 考虑

- 如何**保存**文法符号的综合**属性值**？
- 保存属性值的**数据结构**怎样与**分析栈**相联系？
- **怎样保证**：每当进行归约时，由栈中正在归约的产生式右部符号的属性值计算其左部符号的综合属性值。



修改分析栈

目的：使之能够保存综合属性

做法：在分析栈中增加一个域，存放综合属性值

例：带有综合属性域的分析栈

- 栈：一对数组state和val

- state元素：指向LR(1)分析表中状态行的指针（或索引）

- 如果state[i]保存对应符号A的状态，则 val[i]中就存放A的属性值。

top →

Sz	Z.z
Sy	Y.y
Sx	X.x
---	---

state val

- 假设综合属性刚好在每次归约前计算

- $A \rightarrow XYZ$ 对应的语义规则是 $A.a = f(X.x, Y.y, Z.z)$

修改分析程序

■ 对于终结符号

- 其综合属性值由词法分析程序产生
- 当分析程序执行移进操作时，其属性值随状态符号一起入栈。

■ 为每个语义规则编写一段代码，以计算属性值

■ 对每一个产生式 $A \rightarrow XYZ$

- 把属性值的计算与归约动作联系起来
- 归约前，执行与产生式相关的代码段
- 归约：右部符号的相应状态及其属性出栈，左部符号的相应状态及其属性入栈。

■ LR分析程序中应增加计算属性值的代码段

top →	Sz	Z.z	
	Sy	Y.y	
top →	S X	A .a	
	...	...	
	state val		

例：用LR分析程序实现表达式求值

产生式	语义规则
$L \rightarrow E$	<code>print(E.val)</code>
$E \rightarrow E_1 + T$	<code>E.val = E₁.val + T.val</code>
$E \rightarrow T$	<code>E.val = T.val</code>
$T \rightarrow T_1 * F$	<code>T.val = T₁.val * F.val</code>
$T \rightarrow F$	<code>T.val = F.val</code>
$F \rightarrow (E)$	<code>F.val = E.val</code>
$F \rightarrow \text{digit}$	<code>F.val = digit.lexval</code>

代码段

`print(val[top])`

`val[ntop] = val[top-2] + val[top]`

`val[ntop] = val[top-2] * val[top]`

`val[ntop] = val[top-1]`

0	1
	E.v

↑
top

...	S _E	S ₊	S _T
	E.v		T.v

↑
ntop

↑
top

栈指针变量 **top** 和 **ntop** 的控制：

- 当用 $A \rightarrow \beta$ 归约时，若 $|\beta|=r$ ，在执行相应的代码段之前，
`ntop = top - r + 1`。
- 在每一个代码段被执行之后，`top = ntop`

例：用LR分析程序实现表达式求值

产生式	语义规则
$L \rightarrow E$	<code>print(E.val)</code>
$E \rightarrow E_1 + T$	<code>E.val = E₁.val + T.val</code>
$E \rightarrow T$	<code>E.val = T.val</code>
$T \rightarrow T_1 * F$	<code>T.val = T₁.val * F.val</code>
$T \rightarrow F$	<code>T.val = F.val</code>
$F \rightarrow (E)$	<code>F.val = E.val</code>
$F \rightarrow \text{digit}$	<code>F.val = digit.lexval</code>

代码段

(分析表见表4-8)

`print(val[top])`

`val[ntop] = val[top-2] + val[top]`

`val[ntop] = val[top-2] * val[top]`

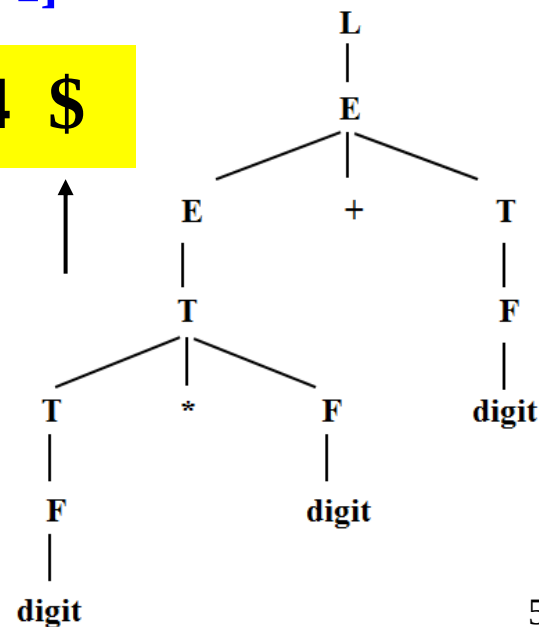
`val[ntop] = val[top-1]`

3 * 5 + 4 \$

state	0	1	6	9
val	-	19		4

↑ ↑ ↑ ↑

accept!



对 $3*5+4$ 进行分析的动作序列 (分析表见表4-8)

步骤	输入	分析栈	分析动作
(1)	$3*5+4\$$	state: 0 val: -	移进 5
(2)	$*5+4\$$	state: 0 5 val: - 3	归约, 用 $F \rightarrow \text{digit}$ goto[0,F]=3
(3)	$*5+4\$$	state: 0 3 val: - 3	归约, 用 $T \rightarrow F$ goto[0,T]=2
(4)	$*5+4\$$	state: 0 2 val: - 3	移进 7
(5)	$5+4\$$	state: 0 2 7 val: - 3 -	移进 5
(6)	$+4\$$	state: 0 2 7 5 val: - 3 - 5	归约, 用 $F \rightarrow \text{digit}$ goto[7,F]=10
(7)	$+4\$$	state: 0 2 7 10 val: - 3 - 5	归约, 用 $T \rightarrow T * F$ goto[0,T]=2

对 $3*5+4$ 进行分析的动作序列

作业：

5.1

5.4

步骤	输入	分析栈	分析动作
(8)	+4\$	state: 0 2 val: - 15	归约, 用 $E \rightarrow T$ goto[0,E]=1
(9)	+4\$	state: 0 1 val: - 15	移进 6
(10)	4\$	state: 0 1 6 val: - 15 -	移进 5
(11)	\$	state: 0 1 6 5 val: - 15 - 4	归约, 用 $F \rightarrow \text{digit}$ goto[6,F]=3
(12)	\$	state: 0 1 6 3 val: - 15 - 4	归约, 用 $T \rightarrow F$ goto[6,T]=9
(13)	\$	state: 0 1 6 9 val: - 15 - 4	归约, 用 $E \rightarrow E+T$ goto[0,E]=1
(14)	\$	state: 0 1 val: - 19	接受



5.4 L属性定义的自底向上翻译

- 在自底向上的分析过程中实现L属性定义的翻译
- 可以实现任何基于LL(1)文法的L属性定义
- 可以实现许多（不是全部）基于LR(1)文法的L属性定义
- 方法
 - 5.4.1 移走翻译方案中嵌入的语义规则
 - 5.4.2 直接使用分析栈中的继承属性
 - 5.4.3 变换继承属性的计算规则
 - 5.4.4 改写语法制导定义为S属性定义

5.4.1 移走翻译方案中嵌入的语义规则

- 自底向上地处理继承属性

- 等价变换：

使所有嵌入的动作都出现在产生式的右端末尾

- 方法：

- 在基础文法中引入新的产生式，形如： $M \rightarrow \epsilon$

- M ：标记非终结符号，用来代替嵌入在产生式中的动作

- 把被 M 替代的动作放在产生式 $M \rightarrow \epsilon$ 的末尾

示例

移除如下翻译方案中嵌入的动作：

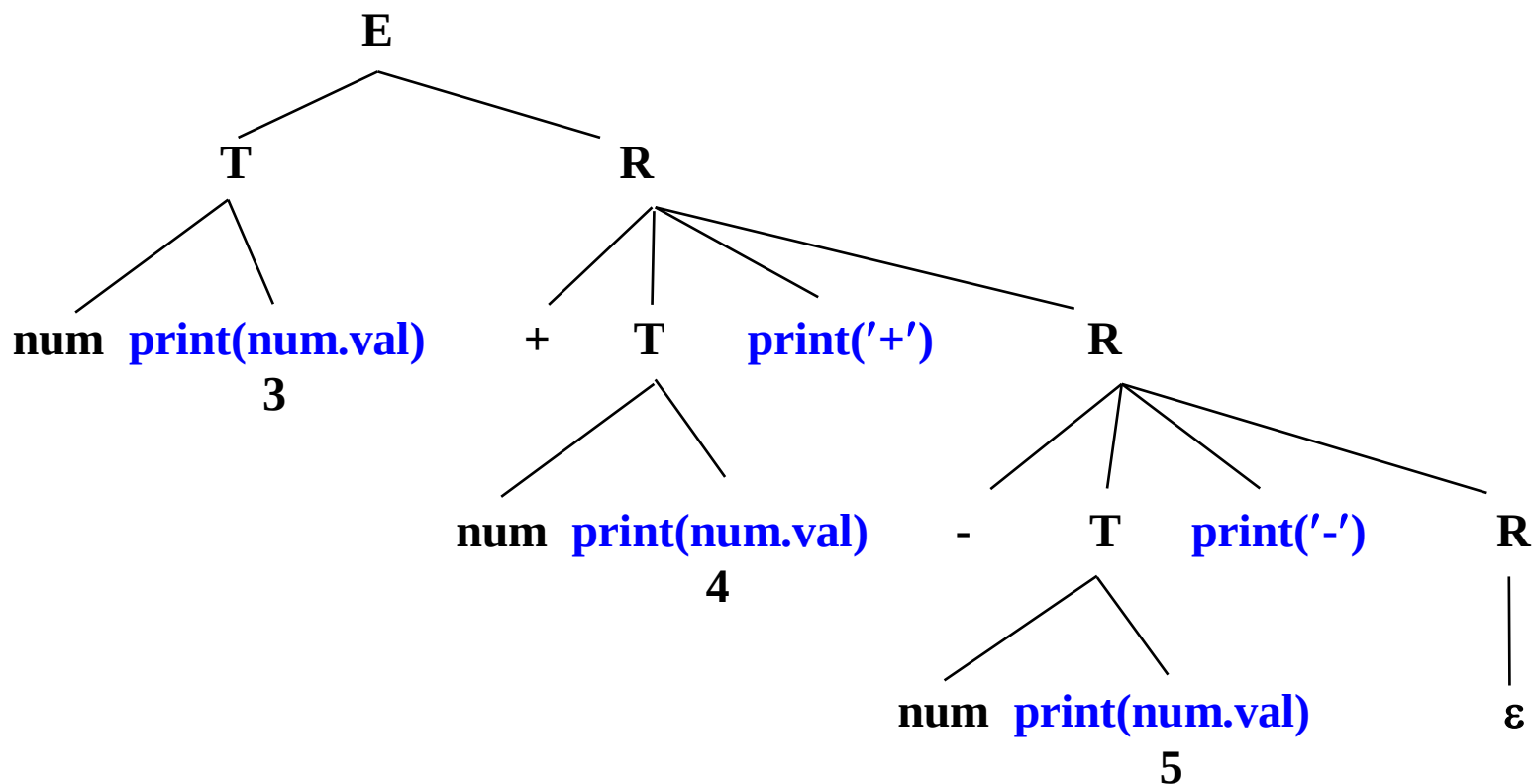
$$E \rightarrow TR$$
$$R \rightarrow +T \{ \text{print}(' + ') \} R \mid -T \{ \text{print}(' - ') \} R \mid \varepsilon$$
$$T \rightarrow \text{num} \{ \text{print}(\text{num.val}) \}$$

- 标记非终结符号M和N，及产生式 $M \rightarrow \varepsilon$ 和 $N \rightarrow \varepsilon$
- 用M和N替换出现在R产生式中的动作
- 被替代的动作放在相应标记非终结符号产生式后面
- 新的翻译方案

$$E \rightarrow TR$$
$$R \rightarrow +TMR \mid -TNR \mid \varepsilon$$
$$T \rightarrow \text{num} \{ \text{print}(\text{num.val}) \}$$
$$M \rightarrow \varepsilon \{ \text{print}(' + ') \}$$
$$N \rightarrow \varepsilon \{ \text{print}(' - ') \}$$

方案等价

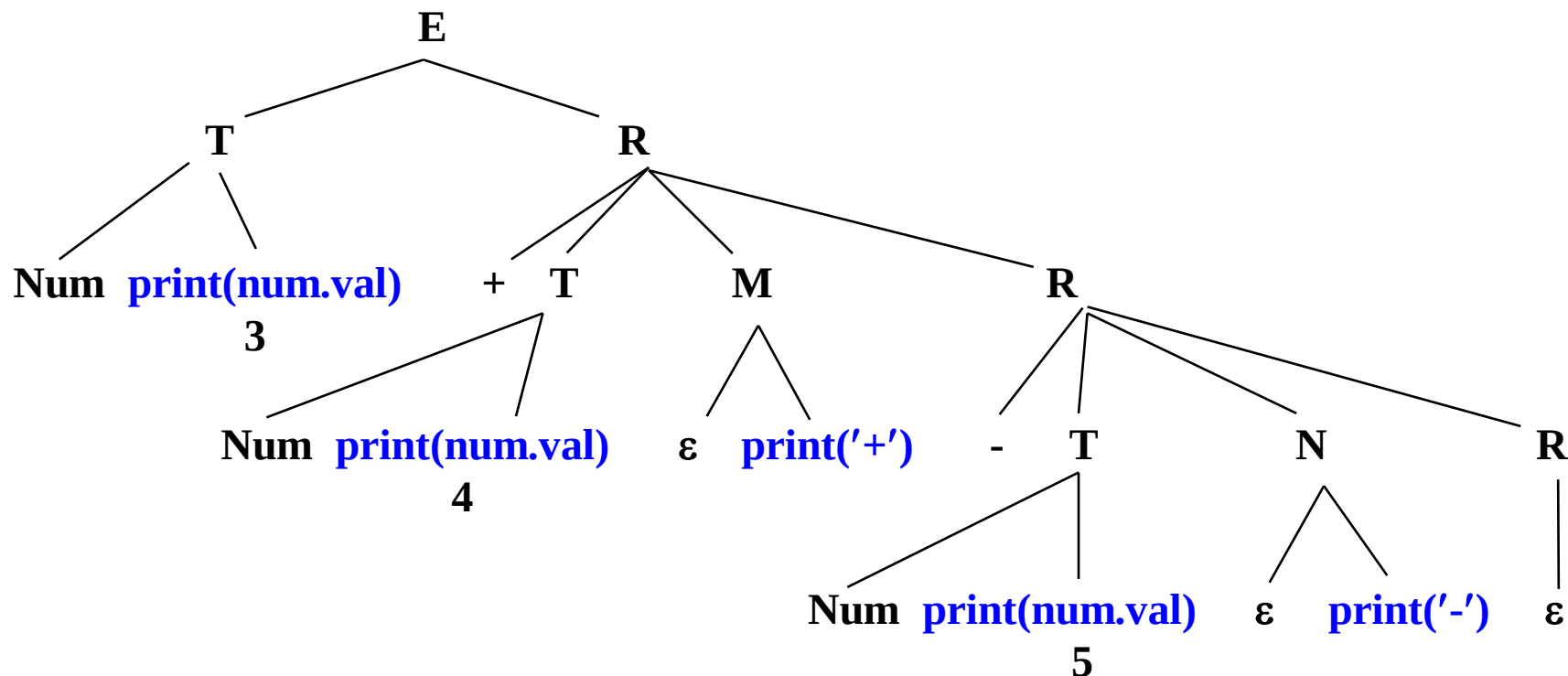
变换前，表达式 3+4-5 的分析树：



- 深度优先遍历
- `print(num1.val) print(num2.val) print('+') print(num3.val) print('-')`
- 动作执行的结果是：34+5-

方案等价

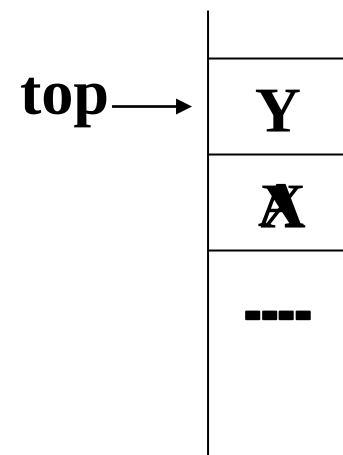
变换后，表达式 3+4-5 的分析树：



- 深度优先遍历
- `print(num1.val) print(num2.val) print('+') print(num3.val) print('-')`
- 动作执行的结果是：34+5-

5.4.2 直接使用分析栈中的继承属性

- LR分析程序对产生式 $A \rightarrow XY$ 的归约
- 考虑分析过程中属性的计算



	Y_k	$Y_k \cdot y$
	X_n	$X_n \cdot x$
	Y_2	$Y_2 \cdot y$
	X_2	$X_2 \cdot x$
	X_1	$X_1 \cdot x$
top →	...	
	state	val

$$X \rightarrow X_1 X_2 \dots X_n$$

$$Y \rightarrow Y_1 Y_2 \dots Y_k$$

$$Y.i = X.s$$

复制规则的重要作用

输入符号串: real p, q, r

翻译方案:

$D \rightarrow T \{ L.in = T.type \}$

L

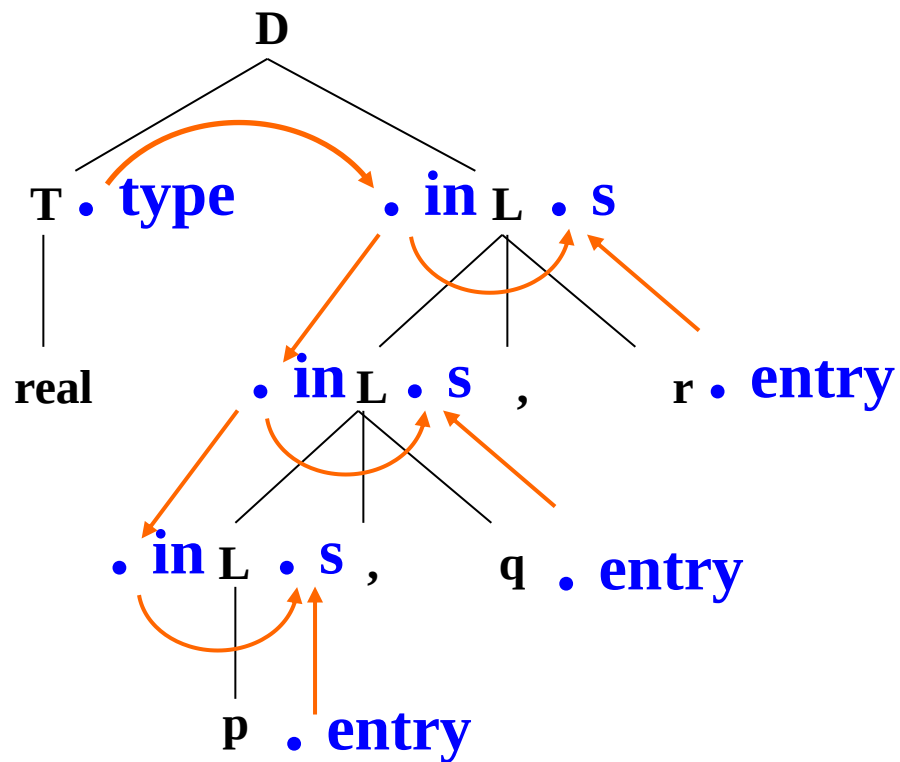
$T \rightarrow \text{int} \{ T.type = \text{integer} \}$

$T \rightarrow \text{real} \{ T.type = \text{real} \}$

$L \rightarrow \{ L_1.in = L.in \}$

$L_1, \text{id} \{ \text{addtype}(\text{id.entry}, L.in) \}$

$L \rightarrow \text{id} \{ \text{addtype}(\text{id.entry}, L.in) \}$



例：应用继承属性，用复制规则传递标识符的类型

输入	栈	分析动作
real p,q,r\$	state: val:	移进
p,q,r\$	state: real val: real	归约, 用 $T \rightarrow \text{real}$
p,q,r\$	state: T val: real	移进
,q,r\$	state: T p val: real pentry	归约, 用 $L \rightarrow \text{id}$
,q,r\$	state: T L val: real -	移进
q,r\$	state: T L , val: real - ,	移进
,r\$	state: T L , q val: real - , qentry	归约, 用 $L \rightarrow L, \text{id}$
,r\$	state: T L val: real -	移进
r\$	state: T L , val: real - ,	移进
\$	state: T L , r val: real - , rentry	归约, 用 $L \rightarrow L, \text{id}$
\$	state: T L val: real -	归约, 用 $D \rightarrow TL$
\$	state: D val: -	接受

addtype(id.entry, L.in)

计算属性值的代码段

产生式	代码段
$D \rightarrow TL$	
$T \rightarrow \text{int}$	<code>val[ntop]=integer</code>
$T \rightarrow \text{real}$	<code>val[ntop]=real</code>
$L \rightarrow L, \text{id}$	<code>addtype(val[top], val[top-3])</code>
$L \rightarrow \text{id}$	<code>addtype(val[top], val[top-1])</code>

- 栈顶指针：top、ntop
- 用产生式 $L \rightarrow \text{id}$ 归约时，L.in 的位置？
- 用产生式 $L \rightarrow L, \text{id}$ 归约时，L.in 的位置？
- 和 L.in 有关的动作？

5.4.3 变换继承属性的计算规则

- 要想从栈中取得继承属性，**当且仅当**文法允许属性值在栈中存放的位置可以预测。
- 例：语法制导定义

	产生式	语义规则
(1)	$A \rightarrow aXZ$	$Z.i = X.s$
(2)	$A \rightarrow bXYZ$	$Z.i = X.s$
(3)	$X \rightarrow x$	$X.s = 5$
(4)	$Y \rightarrow y$	$Y.s = 7$
(5)	$Z \rightarrow z$	$Z.s = g(Z.i)$

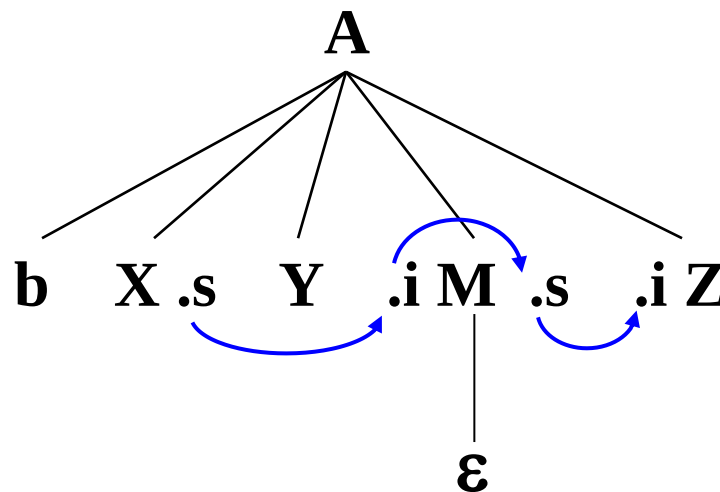
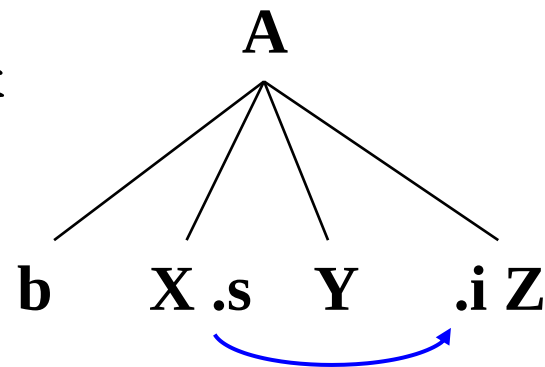
当用 $Z \rightarrow z$ 进行归约时，
 $Z.i$ 可能在 $val[top-1]$ 处
也可能在 $val[top-2]$ 处

属性值在栈中的位置不可预测

模拟继承属性的计算

- 引入标记非终结符号 M 及产生式 $M \rightarrow \varepsilon$
- 设置属性 $M.i$ 和 $M.s$
- 对原语法制导定义进行等价变换

	产生式	语义规则
(1)	$A \rightarrow aXZ$	$Z.i = X.s$
(2')	$A \rightarrow bXYMZ$	$M.i = X.s;$ $Z.i = M.s$
(3)	$X \rightarrow x$	$X.s = 5$
(4)	$Y \rightarrow y$	$Y.s = 7$
(5)	$Z \rightarrow z$	$Z.s = g(Z.i)$
(6)	$M \rightarrow \varepsilon$	$M.s = M.i$



模拟非复制规则的计算

- 例：考虑如下的产生式及语义规则：

$A \rightarrow aXY$ $Y.i = f(X.s)$

$Y \rightarrow y$ $Y.s = g(Y.i)$

...	a	X	y
...		X.s	

↑
top

- 引入标记非终结符号 N 及产生式 $N \rightarrow \epsilon$

$A \rightarrow aXNY$ $N.i = X.s; Y.i = N.s$

$N \rightarrow \epsilon$ $N.s = f(N.i)$

$Y \rightarrow y$ $Y.s = g(Y.i)$

...	a	X	N	y
...		X.s	N.s	

↑
top

- 所有继承属性均由复制规则实现
- 继承属性在栈中的位置可以预测

算法5.3: L属性定义的自底向上分析和翻译

输入: 基础文法是LL(1)文法的L属性定义

输出: 在分析过程中计算所有属性值的分析程序

方法:

假设:

每个非终结符号A都有一个继承属性 $A.i$

每个文法符号X都有一个综合属性 $X.s$

(1) 对每个产生式 $A \rightarrow X_1 X_2 \dots X_n$

引入n个新的标记非终结符号 $M_1, M_2, \dots, M_n$

用产生式 $A \rightarrow M_1 X_1 M_2 X_2 \dots M_n X_n$ 代替原来的产生式

X_j 的继承属性与标记非终结符号 M_j 的综合属性相联系

属性 $X_j.i (=M_j.s)$ 总是在 M_j 处计算, 且发生在完成 X_{j-1} 入栈之后, 开始分析 X_j 的动作之前。

算法5.3

(2) 在自底向上分析过程中，各个属性的值都可以被计算出来

第一种情况：用 $M_j \rightarrow \varepsilon$ 进行归约

■ 已知：

- 每个标记非终结符号在文法中是唯一的
- M_j 属于哪个形式为 $A \rightarrow M_1 X_1 M_2 X_2 \dots M_n X_n$ 的产生式
- 计算属性 $X_j.i$ 需要哪些属性、以及它们的位置

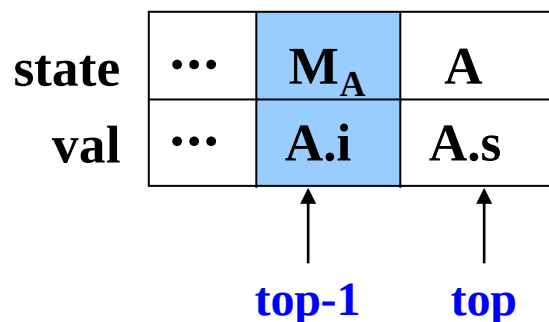
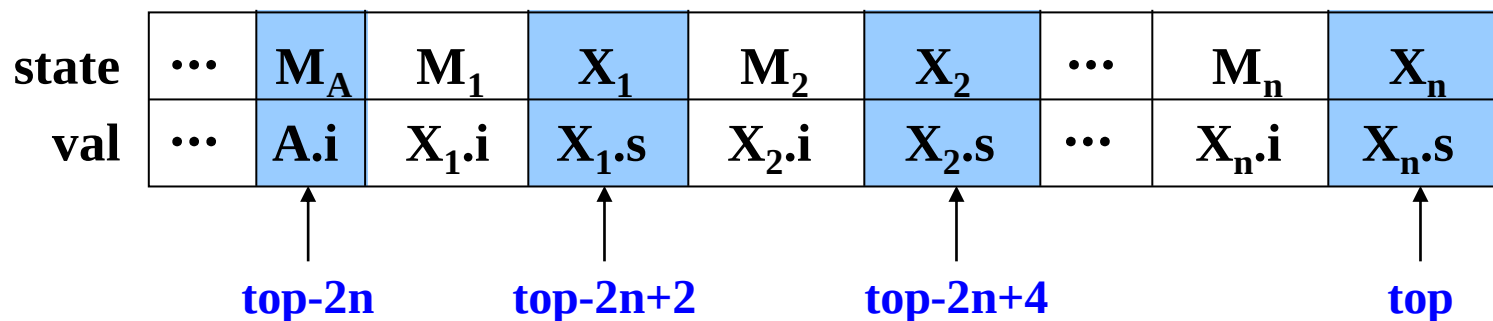
state	...	M_A	M_1	X_1	M_2	X_2	...	M_{j-1}	X_{j-1}	M_j
val	...	$A.i$	$X_1.i$	$X_1.s$	$X_2.i$	$X_2.s$	...	$X_{j-1}.i$	$X_{j-1}.s$	$M_j.s$
		↑	↑	↑	↑	↑		↑	↑	↑
		$top-2(j-1)$	$top-2(j-1)+2$		$top-2(j-2)+2$			top		$ntop$
			$top-2(j-1)+1$	$top-2(j-2)+1$		$top-1$				

算法5.3

第二种情况：用 $A \rightarrow M_1 X_1 M_2 X_2 \dots M_n X_n$ 进行归约

■ 已知：

- $A.i$ 的值、及其位置
- 计算 $A.s$ 所需要的属性值均已在栈中已知的位置
即各有关 X_j 的位置上



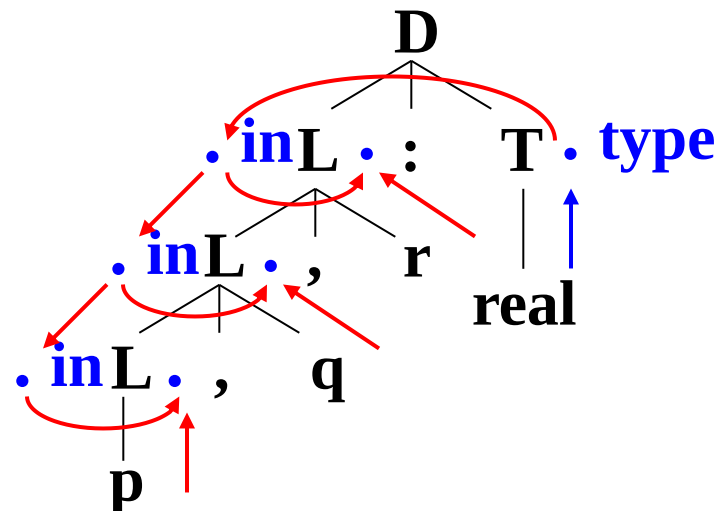
5.4.4 改写语法制导定义为S属性定义

- 例：PASCAL的变量声明语句可由如下文法产生：

$D \rightarrow L:T$

$T \rightarrow \text{integer} \mid \text{real}$

$L \rightarrow L, \text{id} \mid \text{id}$



- 问题：

- 标识符由L产生，而类型不在L的子树中
- 归约从左向右进行，类型信息从右向左传递
- 只用综合属性不能使类型和标识符联系在一起

解决方法

- 改写文法，使类型作为标识符表的最后一个元素

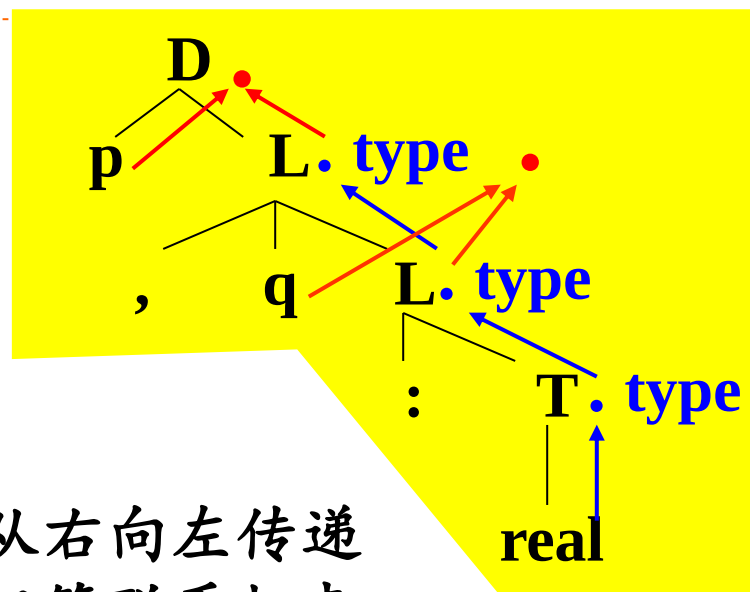
$D \rightarrow idL$

$L \rightarrow ,idL \mid :T$

$T \rightarrow integer \mid real$

- 改写后：

- 归约从右向左进行，类型信息从右向左传递
- 仅用综合属性把类型信息和标识符联系起来



$D \rightarrow idL$

$addtype(id.entry, L.type)$

$L \rightarrow ,idL_1$

$L.type = L_1.type;$

$addtype(id.entry, L_1.type)$

$L \rightarrow :T$

$L.type = T.type$

$T \rightarrow integer$

$T.type = integer$

$T \rightarrow real$

$T.type = real$



本章小结

- 综合属性、继承属性
- 语法制导定义
 - S-属性定义
 - L-属性定义（继承属性应满足的**限制条件**）
- 翻译方案
 - 构造S-属性定义的翻译方案（语义动作放在产生式右尾）
 - 构造L-属性定义的翻译方案（语义动作插入**产生式之中**）
- S-属性定义的自底向上翻译（**改造LR分析程序**）
 - 分析栈的扩充
 - 分析控制程序的修改

本章小结

作业：

5.5 (1)

5.8

5.10

■ L-属性定义的预测翻译 (*)

- 为每个非终结符号构造递归函数
- 形参、返回值、局部变量
- 函数体、分支程序
- 分支程序段的设计

■ L-属性定义的自底向上翻译

- 通过复制规则引用继承属性
- 引入标记非终结符号及相应的产生式
- 修改语义动作、使继承属性由复制规则实现

■ 通用的语法制导翻译方法 (*)

作业(P176)

课堂练习
==>

■ 5.1

■ 5.5 (1)

■ 5.16

■ 5.4

■ 5.8

■ 5.17

■ 5.10



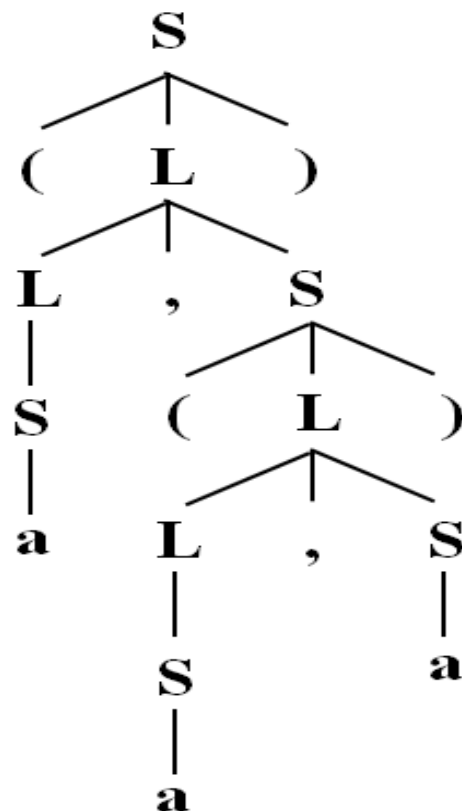
作业 5.16

有如下文法：

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L, S \mid S$$

- (1) 设计一个语法制导定义，它输出配对的括号个数。
- (2) 构造一个翻译方案，它输出每个a的嵌套深度。
如对句子(a,(a,a)),
输出结果是1, 2, 2。

a	(a)	(a,a)	((a),a)
		((a),(a))	((a),(a,(a)))



5.16 (1) 参考答案

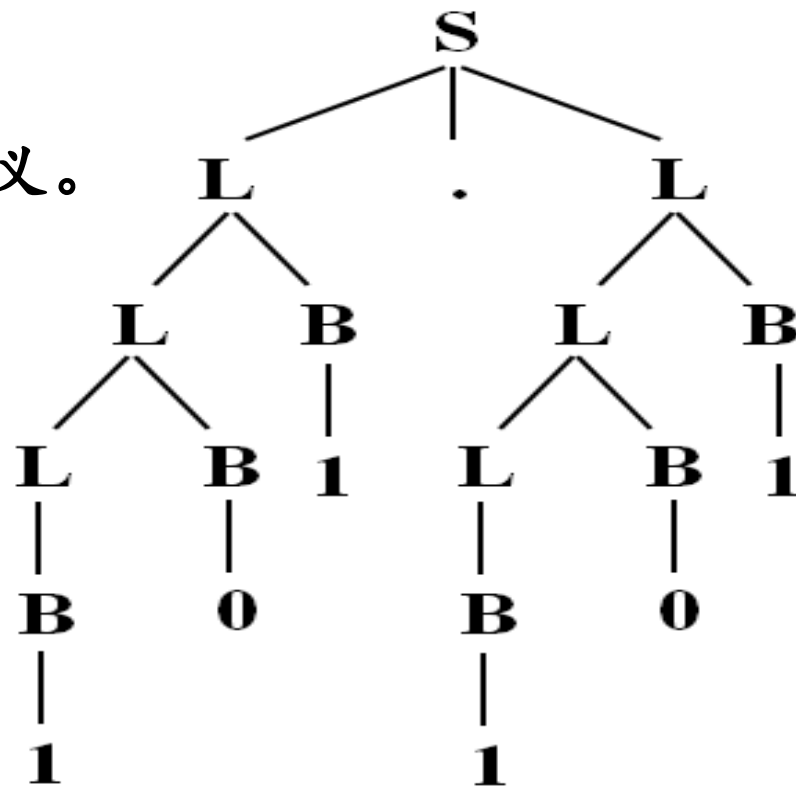
5.16 (2) 参考答案

作业 5.17

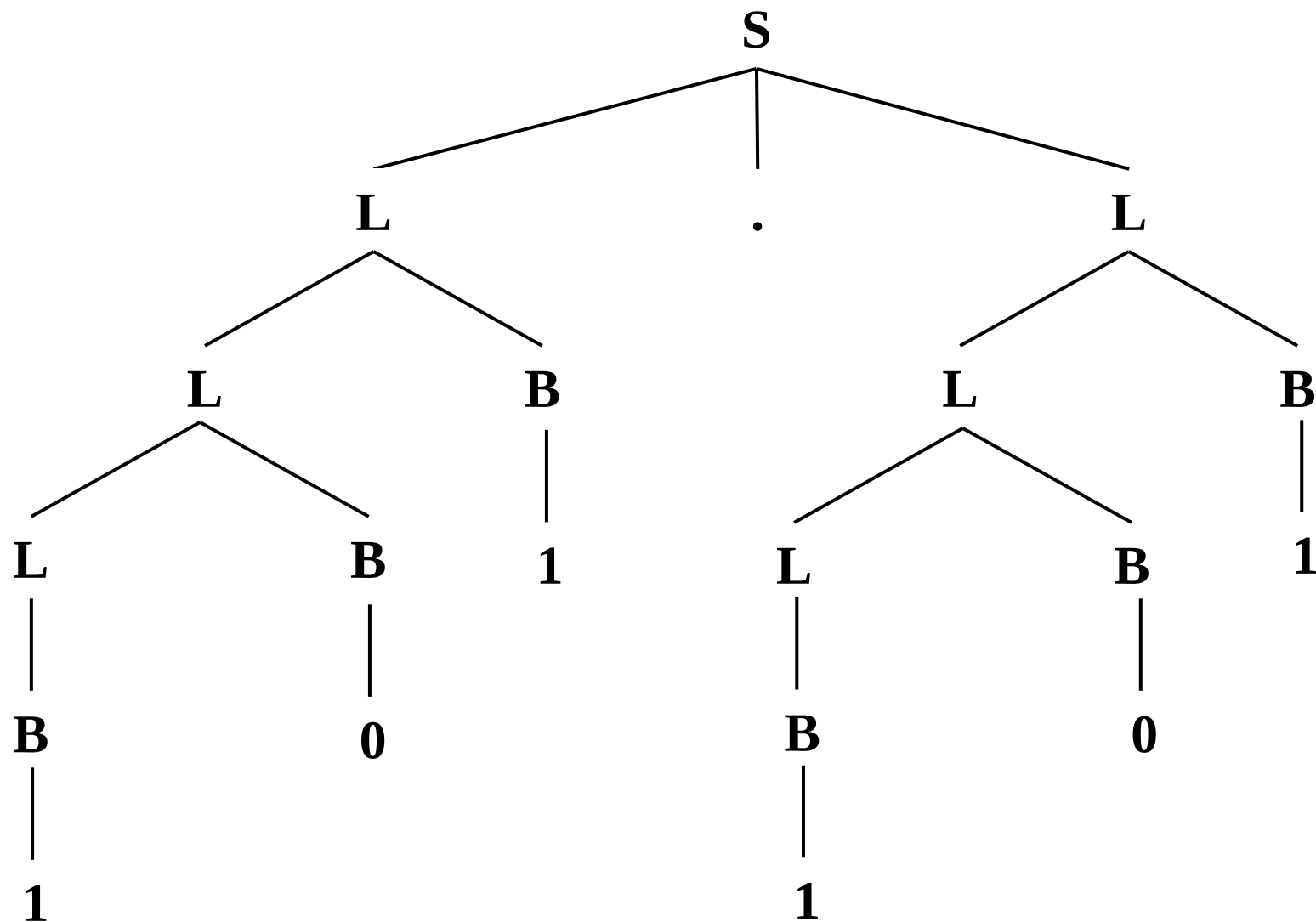
令综合属性 **val** 给出在下面的文法中 **S** 产生的二进制数的十进制数值，如对于输入 101.101， $S.val = 5.625$

$$S \rightarrow L.L \mid L$$
$$L \rightarrow LB \mid B$$
$$B \rightarrow 0 \mid 1$$

请写出确定 $S.val$ 值的语法制导定义。



分析



参考答案

练习

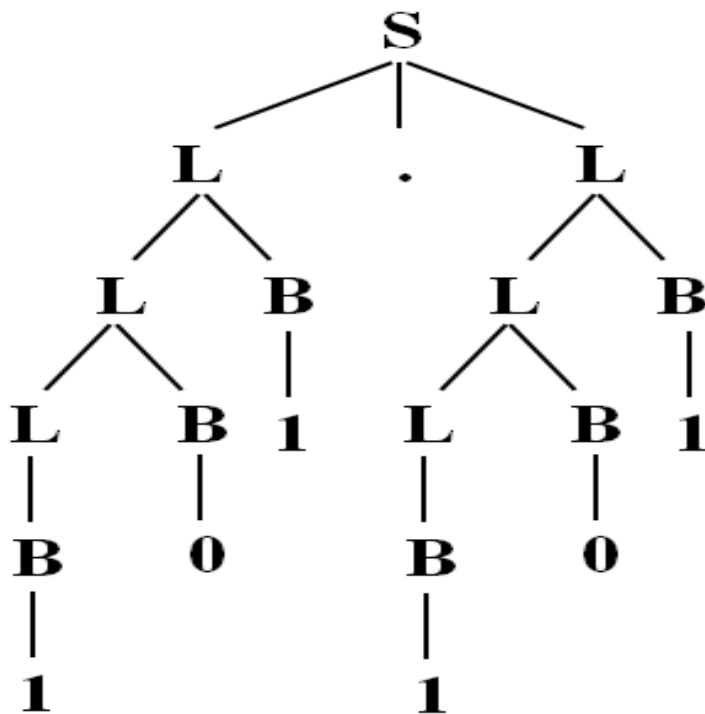
针对如下文法，设计一个翻译方案，打印出输入二进制数中每个1的权值。

如对于输入101.101，打印输出：4，1，0.5，0.125。

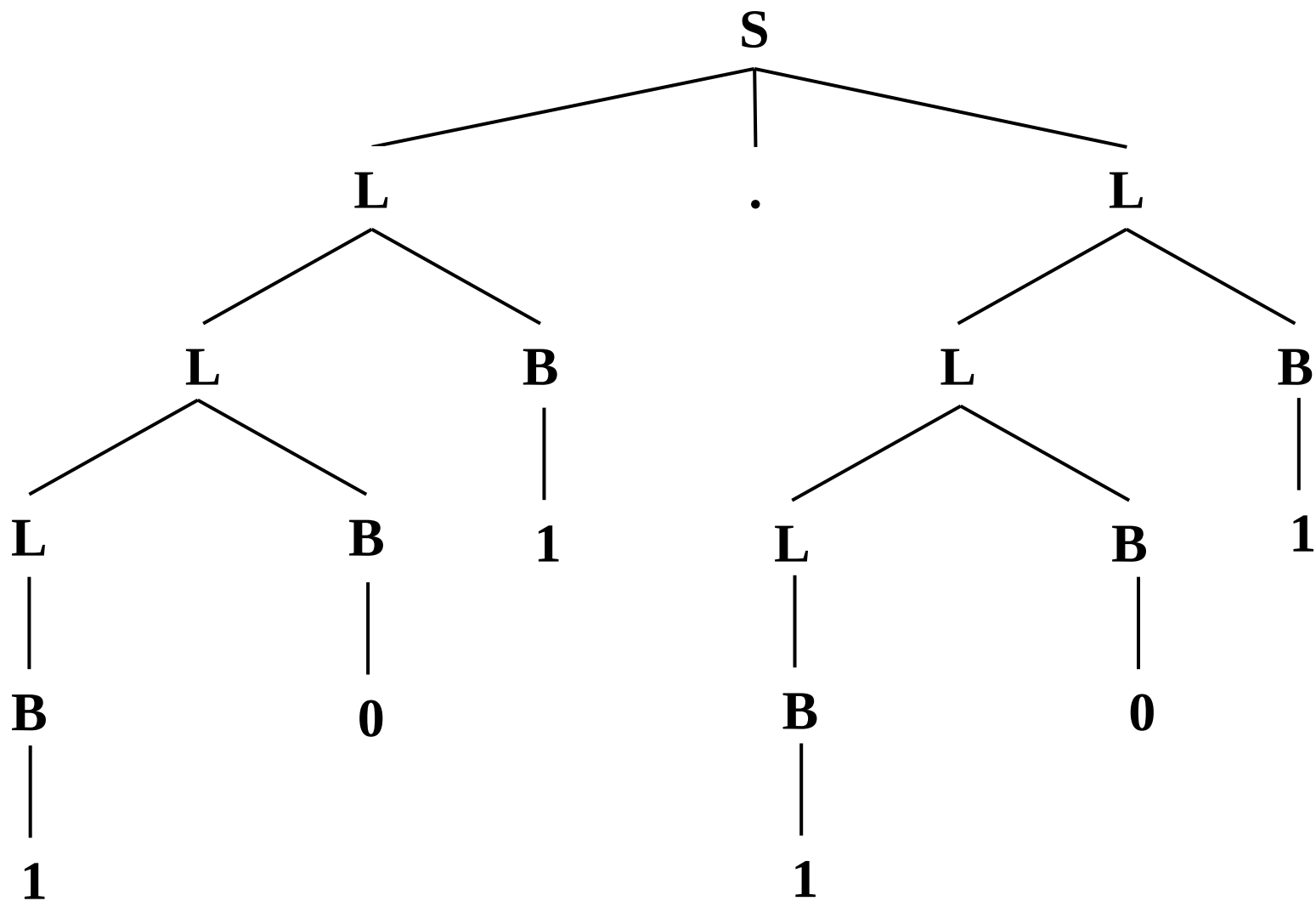
$S \rightarrow L.L \mid L$

$L \rightarrow LB \mid B$

$B \rightarrow 0 \mid 1$



分析



翻译方案

课堂练习1

■ 有文法G[S]:

$$S \rightarrow SaA \mid A$$
$$A \rightarrow AbB \mid B$$
$$B \rightarrow cSd \mid e$$

(1)说明 ebeae**b**ced 是该文法的一个句子;

(2)为该文法设计一个翻译方案, 利用该翻译方案, 可以在自底向上的分析中把上述句子翻译为1314513135246。

ebcedae==>1313524136

参考答案

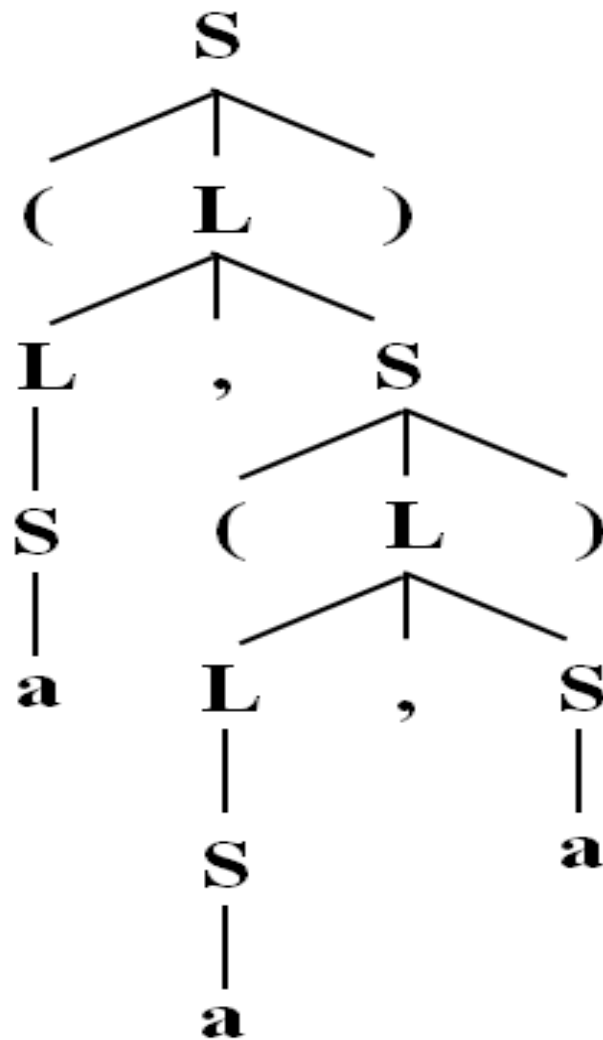
课堂练习2

有如下文法：

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L, S \mid S$$

设计一个翻译方案，使其打印出每个a在输入符号串中的位置。

比如，对于输入符号串(a, (a, a)),
打印输出：2 5 7



参考答案

课堂练习3

有如下文法：

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

设计一个翻译方案，使其打印出每个a在输入符号串中的位置，并统计输出a的个数。

比如，对于输入符号串(a, (a, a)), 打印输出：

第1个a的位置是2

第2个a的位置是5

第3个a的位置是7

一共有3个a。

参考答案

课堂练习4

有如下文法：

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L, S \mid S$$

设计一个翻译方案，使其打印出每个a在输入符号串中的位置,并统计输出a的个数。

比如，对于输入符号串(a, (a, a)), 打印输出：

第1个a的位置是2，嵌套深度是1

第2个a的位置是5，嵌套深度是2

第3个a的位置是7，嵌套深度是2

字符串长度为9，一共有3个a。

参考答案